Game-Evolver: Dual-Loop Evolution Elicits Better End-to-End Game Orchestration

Xucong Wang[1,2*], **Xinghan Chen**[2], **Yundong Zhang**[2], **Weidong Zhang**[2§]

[1]University of Science and Technology of China [2]AI Platform Dept (AIPD), Tencent

[*]Work done during internship at Tencent, [§]Corresponding Author

Abstract

Recent language agents have made significant strides in acting on execution feedback and handling long-horizon tasks, but even frontier models still solve only a minority of executable game-building benchmarks. A game is judged not by whether its code compiles, but by whether it imports, launches, and responds under a real engine contract. Meeting these criteria is already a stiff hurdle. Yet even after that threshold is cleared, quality has no fixed metric and demands costly interactive runs whose outcomes often fluctuate. Writing code that compiles is one challenge, but producing a game that behaves reliably under repeated evaluation is a separate and deeper one. That gap between static code and dynamic execution is where the true bottleneck lies.

We address this bottleneck with Game-Evolver, a dual-loop framework that co-evolves two agents in a dual-loop matrix. A game orchestration agent (GOA) and a harness proposal agent (HPA) drive the two loops in tandem: In the inner loop, the GOA A/B-tests each game runtime against a new harness proposed by the HPA. A harness earns admission only when its matched replay passes dual-rubric verification, where hard rubrics enforce legality and soft rubrics gauge quality. Failed candidates are stored in a database and fed into the outer harness-generation loop. There, the HPA summarizes recurring failures encountered with its proposed harness, then appends new entries to the harness hub alongside metadata on usage, accuracy, and related statistics. At each epoch boundary, the hub undergoes operations like add, delete, revise, merge and keep, which prune ineffective entries and keep the hub's growth controllable. Our design thus advances both the game-making harness and the outer loop that generates it.

We evaluate Game-Evolver across four game benchmarks and six backbone models. It improves every baseline, delivering several-fold gains on weaker models and up to 30% on the strongest. Beyond game building, the approach transfers to general agent tasks on Terminal-Bench and τ^2-Bench. These results suggest that evolution is most effective where runtime execution exposes a gap between intention and outcome. Looking ahead, adaptive dynamic rubrics and the outer loop's own self-evolution stand out as promising directions for future work.

Github: https://github.com/xuc865/Game-Evolver

1 Introduction

Two lines of game agents. Recent language agents have moved from producing isolated answers to acting in dynamic environments, where they plan, call tools, observe state, and react to execution feedback [17, 25, 39, 62, 72, 79]. In games, this capability has been applied along two lines. Game-playing and embodied agents act inside an environment whose rules, state, and reward are already defined [4, 8, 38, 51, 52, 55, 56];

Figure 1 Game-Evolver couples a game orchestration agent (GOA, upper in the figure, shown as a farmer) and harnessproposal agent (HPA, lower in the figure, shown as an inspector). A GOA creates runnable games with real engineswhile receiving iterative updates from the harness proposed by HPA, and the HPA in turn generates new harnesseswhile using the GOA's trajectory to refine its own framework.

computer-use and game-generation systems instead operate tools, workspaces, and engines to construct or modify the environment itself [1, 14, 18, 23, 73]. Both lines run the same plan–act–observe loop, but a game-playing agent searches for behavior in a world that already exists and gets feedback immediately, whereas a game-generation agent must first assemble a valid world before its behavior can be observed or evaluated. The second line is the hard one. A successful result is not merely a program or a collection of assets: the code, resources, scenes, and runtime behavior must agree under a single engine contract, and that agreement becomes observable only after the project is imported, launched, and exercised. Benchmark results confirm that this difficulty is real: on GameCraft-Bench, which stages 140 Godot tasks across 15 game families, even the strongest agent reaches only 41.46% and most agents score below 40% [32]; GameDev-Bench [5] and OpenGame-Bench [18] also reach the similar conclusion Frontier agents therefore still solve only a minority of executable game-building tasks. The bottleneck, we argue, lies not in code-writing ability but in how the agent organizes repeated contact with a heavy interactive runtime.

Two requirements follow. Operationally, this difficulty produces two coupled requirements. ① The agent must produce a runnable game, not just a code-level answer: an omitted task anchor changes what the model edits, while a missing import check or an uncontrolled recovery branch can hide a broken scene or exhaust the budget. ② The system must keep improving, not merely generate once [18]: quality is decided by hard infrastructure validation and by subjective criteria such as playability and aesthetic appeal, so improvement must cover both the produced game and the procedure that creates it. This double goal separates game generation from two nearby settings: game playing, where the world is fixed and a policy is optimized against environment reward [48, 53, 65], and ordinary code generation, where compilation or unit tests provide a narrower correctness signal.

The missing piece: the harness. Meeting these requirements calls for a better procedure, not just a better model. We call that procedure the harness: the instructions, tool interfaces, context construction, recovery logic, and validation rules that determine how the model sees the task and responds to execution feedback [36, 49]. Improving the harness is a separate level of recursive self-improvement (RSI) [26, 29, 39]. Policy-level RSI distills experience into the acting model itself, through reinforcement learning [20, 64] or reflective self-

correction [36, 49]; harness-level RSI instead treats the wrapper around the model as the search object [26, 29, 39]. Harness-level RSI does not transfer directly to game generation, because a runnable game project breaks three assumptions that existing RSI takes for granted:

- A stable, lightweight interface. Existing RSI calls a light interface, such as a compiler or test runner. Game generation instead faces a heavy interactive runtime, where one failed contact makes every later judgment moot. The harness to evolve is therefore a runtime-coupled orchestration layer.
- A canonical oracle. Coding and tool-use RSI is judged by hidden tests or an explicit checker. Games keep such oracles only for hard legality, and quality above that gate is subjective and has no single oracle. Game generation therefore needs fixed checks alongside task-adaptive quality rubrics.
- Cheap attribution. Separating harness benefit from artifact lineage and trajectory luck already needs controlled comparison [30]. Games make it worse: each evaluation is a full interactive run, its outcome is non-deterministic, and averaging over runs is too costly. RSI must therefore use matched replay, comparing parent and child under identical execution conditions.

To deal with these three issues, Game-Evolver couples artifact execution to harness evolution through two agents, a game orchestration agent (GOA) and a harness proposal agent (HPA), that drive the two loops of Figure 1. The inner loop, driven by the GOA, uses the current champion harness to turn a task into an executable game. When the HPA proposes a bounded harness change, the inner loop A/B-tests the game made with the new harness and admits it only if matched replay passes dual-rubric verification; failed candidates are stored in a database and handed to the outer loop. Driven by the HPA, the outer loop summarizes recurring failure families and proposes the next bounded harness change: execution produces evidence, and matched replay decides the fate of that evidence. The ordering of the two loops is essential, because artifact improvement and harness improvement would otherwise be confounded: a successful game may come from a favorable artifact lineage or a lucky trajectory, so it cannot by itself show that the harness improved. Game-Evolver therefore content-addresses harness profiles and freezes a profile for the duration of an episode, changing the champion only when matched replay evidence satisfies the admission rule. The outer loop manages the reusable harness hub that the proposer draws on, so procedural improvements accumulate across epochs without disturbing the profile under evaluation. Our contributions are fourfold:

- We formulate game generation as an executable, budgeted orchestration problem and separate artifact evolution from evolution of the harness that generates artifacts.
- We implement a searchable harness profile covering the core of the game-making loop, such as its instructions, tool interfaces, context compiler, and recovery and validation policies.
- We implement a self-evolution loop in which the inner loop improves the game-agent harness and the outer loop manages the reusable knowledge that shapes future proposals.
- Extensive experiments on four game-generation and two common benchmarks validate the superiority of our model, which delivers several-fold gains on weaker models and up to 30% on the strongest.

2 Related Work

2.1 Game AI and Game Orchestration Agents

The classic game-AI line treats a game as an environment whose transition dynamics, action space, and success signal are already available, and asks an agent to learn behavior within that fixed world. Deep Q-learning made this setting concrete on Atari by mapping observations to actions under delayed reward, while AlphaGo [51], AlphaGo Zero [52], and AlphaStar [55] combined search, self-play, and learned policies to scale the same formulation to increasingly difficult control problems [38]. OpenAI Five [4] extended it to a larger multi-agent arena in which coordination and long horizons matter, but the game rules and executable environment remain external to the agent. Later work relaxed the assumption that behavior must be a single reactive policy. Video PreTraining learns reusable behavior from large-scale gameplay traces, MineDojo and Voyager add open-ended objectives, memory, and skill reuse in Minecraft, and Generative Agents study persistent behavior in a simulated social world [2, 8, 42, 56]. Cradle [54] and MobileAgent-v2 [57] extend the

same plan–act–observe pattern to richer graphical and device-control settings, where an agent must track state and recover across many interface actions. Across these variants, the agent becomes more general and the execution loop becomes longer, yet the environment in which the loop runs is still assumed to exist before evaluation starts.

While that line of work is close to our setting, as mentioned above, it still assumes that an environment already exists when evaluation begins. Recent computer-use systems push the same control view into graphical and desktop interfaces [11, 13, 28, 77], while benchmarks like ALFWorld [50] and AndroidWorld [45] make long-horizon interaction, state tracking, and recovery explicit [34]. Game-Evolver starts one step earlier: the candidate environment must first be made runnable, and the object of improvement is the harness that organizes that construction process.

2.2 Game-Generation Benchmarks

Game generation builds on web, GUI, and computer-use suites that score actions in stateful environments beyond static answers [34, 35, 59–61, 67, 70, 81]. Recent game generation benchmarks, such as OpenGame-Bench, score games over 150 prompts along build health, visual usability, and intent alignment [18]; GameCraft-Bench [32] stages 140 Godot tasks across 15 game families and replays recorded traces in a Godot runtime; it then scores hidden rubrics over core mechanics, content depth, functional visuals, and art presentation; and GameDev-Bench raises the difficulty to multi-file Godot tasks that must remain importable and pass deterministic execution-based checks [5]. Web-game and Pygame suites extend the same principle to lighter runtimes: VeriGame evaluates web games through a verifier chain with runtime state injection over 100 games in seven genres [15], while V-GameGym scores 2,219 Pygame samples across 100 thematic clusters by grading code, screenshots, and video together over a fixed recording horizon, so visible gameplay, beyond a static patch, is required [80]. The same standard now reaches heavier and more interactive runtimes: AutoUE generates 3D games in Unreal Engine through a multi-agent system with automated play-testing [73], and PlaytestArena pairs 200 browser-based generation tasks across eight genres with rubrics of expected in-play behavior, letting a GUI agent load each build, play it, and adjudicate rubric compliance [14]. Validation has even become a benchmark target of its own: GBQA injects 124 human-verified bugs into 30 games and finds that a frontier model detects fewer than half of them, confirming that checking a running game is harder than generating its code [16]. Across all of them, evaluation moves from code to runnable artifact, where a harness governs how a candidate is inspected and repaired before it is validated.

2.3 Agent Self-Evolution

Parameter-level evolution. Here parameter-level evolution denotes changes to the agent-side search object, including model weights, prompts, policies, reward functions, and searched programs. Some methods like APE [82] and OPRO [68] treat instructions or pipelines as parameters that can be proposed, scored, and revised [9, 10, 21, 76]. Self-Consistency and Self-Refine improve inference-time traces, while Self-Rewarding and Math-Shepherd use model-generated evaluation to select or supervise stronger reasoning behavior [36, 58, 63, 75]. At the weight level, PPO, RLHF, and direct preference optimization convert feedback into updated model parameters [6, 31, 41, 44, 48, 74]; Search-R1 and RAGEN extend this idea to multi-turn experience [20, 64]. Related program-search systems modify another agent-side object: Eureka searches reward code, FunSearch searches programs, and AutoML-Zero and AlphaEvolve broaden the search toward general algorithms [33, 40, 46, 47]. Although these methods optimize different parameterizations, the improved object is still highly limited within an externally specified harness.

Harness-level evolution. A second line changes the harness itself: the roles, instructions, tool interfaces, context construction, recovery branches, and validation rules that surround the model. Multi-agent systems such as CAMEL [27], AutoGen [66], MetaGPT [12], and ChatDev [43] expose this organizational layer through specialized roles and handoffs, while Multi-Agent Debate makes the resulting execution graph explicit [7]. More recent harness-centric work [78], including Code as Agent harness [39], Meta-harness [26], and Observability-Driven Automatic Evolution [29], treats the wrapper as a measurable and adjustable search object rather than a fixed implementation detail; AgentX pushes the same direction further [24]. Code and computer-use benchmarks provide the external execution signals needed to judge such changes [19, 22, 69, 81]. Game-

Evolver belongs to this second line. It evolves the harness that inspects, repairs, and validates an executable game artifact, while keeping the backbone parameters, evaluator, seed, and admission rule fixed during each matched comparison. This separation lets a measured gain be attributed to the harness around the model rather than to a simultaneous update of the model itself.

3 Problem Setting

The preceding discussion separates two objects: the runnable game artifact and the harness that produces it. We first formalize the artifact-level task and the evidence returned by execution; Sections 4 and 5 then describe how Game-Evolver improves the artifact and its producing harness without conflating the two.

3.1 Task Definition

We formalize the general task as constructing a runnable game project from a task specification and an initial project state. Because that initial state may be either a template or a partially implemented project, the same formulation covers end-to-end generation and repair without assuming a particular agent architecture. Let $x \sim \mathcal{D}$ denote a task with natural-language requirements r_x, an initial project $p_{0,x}$, an engine/runtime configuration γ_x, and a task-specific evaluation protocol $\mathcal{E}_x$. The project may span code, content, interaction bindings, and configuration, all of which must remain valid under the same engine contract when the task is complete. We thus target a game project characterized by

$$p_x^\star \in \arg\max_{p \in \mathcal{P}(p_{0,x},\gamma_x)} S_x(p) \quad \text{subject to} \quad F_x(p) = 1, \qquad Q_{x,k}(p) \geq \ell_{x,k} \quad (k = 1, \ldots, m_x), \tag{1}$$

where $\mathcal{P}(p_{0,x}, \gamma_x)$ is the set of projects reachable from the initial state while respecting the engine configuration, and m_x is the number of explicit auxiliary criteria. The primary score S_x, feasibility indicator F_x, criterion values $Q_{x,k}$, and thresholds $\ell_{x,k}$ are supplied by the task evaluator; when a benchmark exposes no auxiliary constraint, this set is empty. Their meanings are consequently task-specific, tied to the benchmark that defines them.

3.2 Executable Interaction and Verification

The project space implies multi-step synthesis, because a project is reached through stateful interaction that unfolds over many model responses. At step t, let p_t be the current project state, z_t the observable runtime or tool state, and a_t an admissible interaction. The resulting development trajectory is

$$(p_0, z_0) \xrightarrow{a_0} (p_1, z_1) \xrightarrow{a_1} \cdots \xrightarrow{a_{T-1}} (p_T, z_T), \qquad a_t \in \{\text{inspect, edit, run, interact, observe}\}. \tag{2}$$

Each action can modify several files or resources, and the resulting state can therefore expose a problem that was invisible at the previous step. A locally plausible edit may later break an import or a scene, or regress an interaction, which is why tool outputs and runtime observations must be treated as development evidence. The submitted object is nevertheless the final executable project p_T, separate from the trajectory itself.

In contrast with searching or retrieval, game generation demands more than file existence or successful parsing: the final project must be imported, launched, and exercised under the task's engine and interaction conditions. The resulting runtime evidence ξ_T is passed to the task evaluator through

$$p_T \xrightarrow{\text{import + launch + interaction}} \xi_T \xrightarrow{\mathcal{E}_x} o_x = \big(S_x(p_T), \mathbf{Q}_x(p_T), F_x(p_T), \mathbf{D}_x\big). \tag{3}$$

The output combines the primary score, task-specific objectives or constraints, feasibility, and diagnostics; in particular, $\mathbf{D}_x$ contains evidence references while $\mathbf{Q}_x$ records the task-specific quantities. Correctness therefore belongs to a project that runs in an engine and responds to interaction.

4 Inner-Loop Evolution for the Game Orchestration Agent

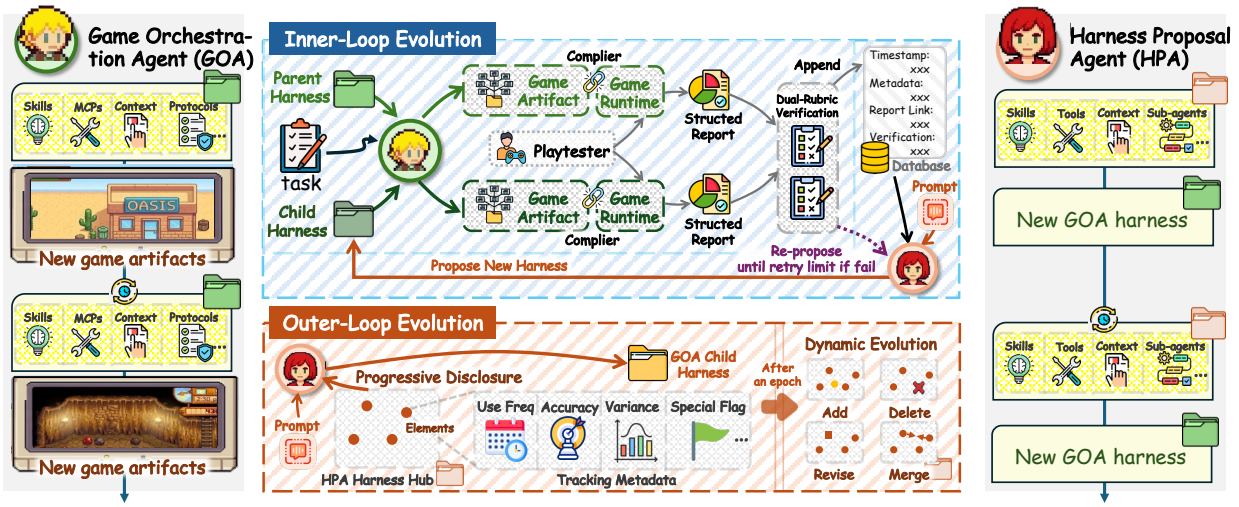

Figure 2 Game-Evolver is a dual-loop evolution framework. It leverages a game orchestration agent (GOA) to create executable games and a harness proposal agent (HPA) to elevate its harness. The inner loop conducts an A/B test of the original and new harnesses by comparing the artifacts that each GOA run generates, with dual-rubric verification for comprehensive judgment.

4.1 Loop Components

While Section 3 defines the executable target and the evidence produced by evaluating it, the inner loop in Game-Evolver holds the game-making harness fixed within an episode, improves the game artifact under that harness, and records the evidence needed to compare harness later.

To be concrete, the game-making harness is the interface between the backbone model and this task; Figure 2 provides an overview of the framework that couples the inner loop to the outer one. Its contents are loaded via progressive disclosure: the agent first sees only the harness profile, and when it decides to use the harness it loads the full context through the `harness.view` tool. Concretely, the profile is $H = (\mathcal{M}, \mathcal{U}, \pi_{\text{ctx}}, \pi_{\text{rec}}, \pi_{\text{val}}, \mathcal{Z})$, where each component plays a distinct role in the game-making harness. Specifically, $\mathcal{M}$ groups the instruction modules that steer the agent's planning and runtime behavior; $\mathcal{U}$ bounds the agent to a fixed set of engine and workspace interfaces; π_{ctx} compiles the task anchors and the evidence gathered during an episode into the context; π_{rec} prescribes bounded retries and isolated recovery on infrastructure failures; π_{val} fires the replay and rubric gates and decides when a targeted repair branch is entered; and $\mathcal{Z}$ pools the reusable execution assets, such as seeded catalog elements, that an episode may draw on. Because the profile is content-addressed, every episode can identify the exact harness that produced its artifact.

The loop therefore begins at a benchmark adapter, before any unconstrained model call. The adapter stages the public task inside a candidate workspace, compiles the task context, and defines the artifact path before the model is invoked. The execution contract is shared across GameCraft-Bench, GameDev-Bench, and other adapters; the seed artifact is staged, hashed, and checked by the artifact gate before any external call.

Once the workspace is prepared, it enters the OpenGame runtime, where the staged task becomes a live interaction. The runtime injects the selected backbone and the episode-frozen harness into context, exposes bounded workspace and engine tools, and returns observations after each action. These interfaces let the agent request evidence-first planning, observability, and bounded repair; they also let it carry failure and probe summaries forward, retry a bounded infrastructure failure, or enter validation repair when a gate produces an actionable signal.

The runtime's candidate then crosses a layered evaluation boundary, and that boundary is what keeps the loop honest. Workspace gates first enforce scope and structural constraints, together with suspicious-reference checks. Disposable-copy probes next expose import, engine, or gameplay diagnostics. Only after those checks does the official evaluator report feasibility, objectives, constraints, and the primary score. This ordering lets probes guide repair while keeping the benchmark evaluator as the authority on success.

4.2 Loop Evolution

The inner loop alternates between two activities. With the execution pipeline of the previous subsection in place, the controller improves the game artifact while the harness stays frozen; this is the game-making run. The loop itself evolves only when the HPA proposes a child harness, at which point the GOA repeats the game-making run under the child harness on the same cases, and the two artifact sets are compared. We describe the run first, then the comparison.

Game-making run under a fixed harness. At generation g, the controller reads the champion artifact y_g, its normalized observation o_g, and the preceding trajectory, and selects a benchmark-independent mutation intent m_g. The adapter and context compiler make that intent executable:

$$(y_g, o_g, \tau_{\le g}) \xrightarrow{\text{mutation policy}} m_g \xrightarrow{\text{adapter + context compiler}} \zeta_g = (x, y_g, m_g, \text{feedback}). \tag{4}$$

The compiled context ζ_g is then executed under the fixed backbone and episode-frozen harness. The resulting artifact is checked and, according to the selection policy, either becomes the next champion or leaves the current champion unchanged:

$$\zeta_g \xrightarrow{\text{fixed } M,\text{frozen } H} (y'_g, \tau'_g) \xrightarrow{\text{gate + probes + official evaluator}} o'_g \xrightarrow{\text{selection policy}} (y_{g+1}, o_{g+1}) \text{ or } (y_g, o_g). \tag{5}$$

The intent may target improvement, constraint repair, or an alternative strategy. It remains benchmark-independent because the adapter compiles it into task-specific context while the mutation policy itself stays general. This internal loop is what we mean when we say the GOA makes a game under a harness: the harness shapes how the model plans, edits, and reacts to runtime feedback at every generation.

Harness A/B: parent versus child. When the HPA proposes a child harness $\widetilde{H}^{\text{in}}_{t+1}$, the inner loop re-runs this game-making process under the child harness through **matched replay**, i.e., the same held-out cases, the same seeds, and the same execution conditions that produced the current champion H^{in}_t. The two runs therefore yield two artifacts per case, one made under the parent harness and one under the child harness, and their outcomes o_c^- and o_c^+ differ only in the harness. The loop then feeds the two artifact sets to the dual-rubric verification of Section 4.3, which decides whether the child harness is admitted or the parent remains the champion. Once the champion is decided, the GOA continues to update the game artifact under the winning harness, so artifact improvement and harness improvement alternate rather than overlap. Our design keeps the two sources of gain separable: a better game can come from a better harness or from a better artifact lineage, and only the matched comparison isolates the harness contribution.

4.3 Verification

Given the parent and child artifact sets produced by the matched replay of Section 4.2, verification decides whether the child harness is admitted. A verification stage first rejects unsafe or malformed outputs before internal scoring and runs engine probes to assess runtime health; if either the parent or the candidate artifact fails this initial check, the admission is recorded as a failure, preserving fairness and safety.

Once the parent and child artifacts are validated as legal, they enter the dual-rubric A/B verification pipeline, illustrated in Figure 3. Rubrics split along two orthogonal dimensions. Along the enforcement dimension, hard rubrics measure the legality of game execution and output binary 0/1 values, whereas soft rubrics score content completeness, subjective experience, and other non-legality attributes as continuous values in $[0, 1]$ from an LLM judge. Along the provenance dimension, fixed rubrics are dataset-agnostic criteria that stay fixed throughout evolution, whereas dynamic rubrics are generated from the task description and the evidence surface exposed by the current champion harness. For each parent-candidate comparison, the resulting rubric definitions and weights are instantiated before scoring and shared by both artifacts. Hard rubrics are always fixed, and the remaining soft-rubric weight is split between fixed and dynamic forms, yielding an overall allocation of 30% soft-dynamic, 30% soft-fixed, and 40% hard-fixed weight.

Our design arises from a deliberate trade-off. If every rubric were regenerated from the newest harness and task at each step, a model could draft criteria that flatter the current candidate, and verification would drift toward the local optimum of whichever task happens to be active. If every rubric were frozen instead, the

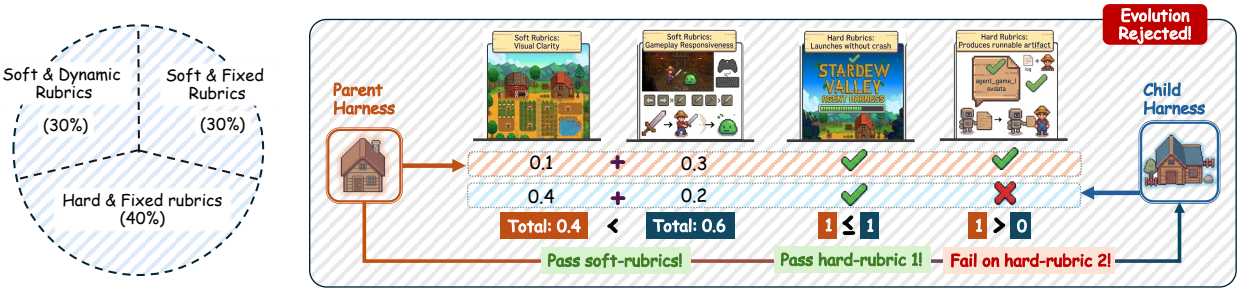

Figure 3 Dual-rubric verification for internal loop. **Left:** The rubrics consist of soft & dynamic rubrics, soft & fixed rubrics, and hard & fixed rubrics. Hard rubrics measure the runnability and legality of game artifacts; soft rubrics measure the completeness and other attributes of the game. Fixed rubrics are dataset-agnostic and fixed alongside the evolution; dynamic rubrics are generated from the parent harness and task descriptions. **Right:** A new harness is accepted only if it matches or exceeds the parent harness on every hard rubric and in the sum of all soft rubrics.

evaluation could neither surface the fine-grained improvements that a particular candidate introduces nor escape an excessive emphasis on the held-out set. Keeping hard rubrics fixed guards the legality gate against redefinition and anchors a stable 40% hard-rubric block, while the dynamic share is confined to soft rubrics so that newly observed behavior can be captured without allowing legality itself to shift.

Formally, let o_c^- and o_c^+ denote the parent and child outcomes for case c, and let $\mathcal{C}_{\text{valid}}$ contain the aligned, feasible, score-bearing pairs with matching budgets and no infrastructure failure. Let $\mathcal{R}_h$ and $\mathcal{R}_s$ index the hard and soft rubrics, with w_j the nonnegative weight of soft rubric j. The comparison selects held-out game tasks and evaluates the two rubric families separately:

$$\forall c \in \mathcal{C}_{\text{valid}}: \qquad \mathsf{HardGate}(c) = 1 \iff \bigwedge_{j \in \mathcal{R}_h} r_{c,j}^+ \geq r_{c,j}^-, \qquad r_{c,j}^{\pm} \in \{0,1\}. \tag{6a}$$

$$\forall c \in \mathcal{C}_{\text{valid}}: \qquad \mathsf{SoftGate}(c) = 1 \iff \sum_{j \in \mathcal{R}_s} w_j r_{c,j}^+ \geq \sum_{j \in \mathcal{R}_s} w_j r_{c,j}^-, \qquad r_{c,j}^{\pm} \in [0,1]. \tag{6b}$$

A candidate harness is admitted only if $|\mathcal{C}_{\text{valid}}| \geq K$, where K is the minimum number of valid cases, protocol checks pass, and both gates hold for every valid case, with soft-rubric weights fixed within the parent-candidate pair. Consequently, every hard rubric stays non-regressing, while the weighted aggregate of the soft rubrics stays non-regressing as a whole. Dynamic rubric generation may tailor the criteria to a task, although the rubric set and weights remain fixed within each matched pair; it neither changes this admission rule nor replaces the official evaluator.

The ledger entry completes verification by turning every attempt, including a rejected one, into future evidence. It records the salient details of each attempt, such as the artifact hash, harness identity, intent and score change, along with the final status, so later harness epochs can reason from auditable traces, replacing an unstructured memory of what happened.

5 Outer-Loop Evolution for the Harness Proposal Agent

5.1 Loop Components

Section 4 keeps a harness fixed while it produces a game artifact; here we reverse that lens and ask how the harness itself should evolve once its behavior has been observed in the wild. The split is intentionally asymmetric, because the inner loop searches the game-making harness that sits in front of the backbone model, whereas the outer loop searches the reusable element library that supports the proposer responsible for inner changes. In compact form,

$$H_t^{\text{in}} = \langle \mathcal{M}_t, \mathcal{U}_t, \pi_{\text{ctx},t}, \pi_{\text{rec},t}, \pi_{\text{val},t}, \mathcal{Z}_t \rangle, \qquad H_t^{\text{out}} = \langle E_t, S_t, P_t \rangle,$$

where the inner tuple names the game-facing modules of Section 4, namely instruction modules, tool interfaces, the context compiler, the recovery and validation policies, and reusable execution assets, while the outer tuple names the reusable element catalog, its seeded subset, and its mutation policy.

Inner loop: game-orchestration harness		Outer loop: harness-generation harness	
harness type	**Allowed scope and action**	**harness type**	**Allowed scope and action**
Instruction modules	Planning, gameplay, observability, repair, and strategy modules; one epoch may add, drop, or swap one module while the other dimensions remain frozen.	Reusable catalog	*skill*, *MCP*, *tool*, *context*, *protocol*, and *workflow* entries; one epoch may add, prune, merge, or replace reusable entries.
Tool interfaces	Bounded `mcp_server`, `command_wrapper`, `engine_probe`, and `env_binding` interfaces; one epoch may expose or retract one interface under a fixed safety scope.	Seeded active set	Per-category seed elements; one epoch may change which reusable elements are preloaded into the proposer before it writes an inner-harness gradient.
Context compiler	Task anchors, probe summaries, failure traces, and edit history; one epoch may change their ordering, truncation, or inclusion policy.	Active caps	Per-category element budgets; one epoch may raise or lower how many elements of each category can be active together.
Recovery policy	Infrastructure retries and isolated recovery directories; one epoch may adjust the retry budget or the branch-isolation rule.	Mutation policy	Usage-driven removal, rewrite, and merge thresholds; one epoch may bias which stale or low-accuracy elements are changed after enough evidence accumulates.
Validation policy	Replay gates, rubric gates, and targeted repair branches; one epoch may change when a repair branch is entered or when a candidate is rejected.	Library workflows	Add, prune, merge, and replace workflows; one epoch may turn a rejected diagnosis into a reusable update for future inner proposals.

Table 1 The two nested loops expose different harness types. The inner loop edits the game-facing harness that creates runnable artifacts, while the outer loop edits the reusable element library that shapes future inner proposals.

Table 1 lists the harness types each loop may change. While the inner epoch changes how the game is executed, replayed, or repaired, the proposer's reusable library stays fixed; the outer epoch reshapes that library by analyzing inner-loop trajectories, so the next inner proposal inherits that structure cleanly. As a result, the two loops account for different parts of the story: the inner loop explains why a game run improved, and the outer loop why the next proposal became better structured.

The handoff from execution to evolution begins when completed episode records are condensed into a benchmark-neutral report. That report collects outcome counts, repeated quality-failure families, and infrastructure events, and it keeps references to the concrete evidence that produced them. The semantic-gradient proposer then reads that report together with structured rejection memory, so the next candidate is guided by what actually failed before. Whether the implementation uses the deterministic fallback or an external expert agent, the stable JSON interface asks for a diagnosis, target tags, and supporting references, and leaves the fitness score to the replay evaluator. In that sense, the report says what should change, while the replay evaluator still decides whether the change worked.

The evolution engine then turns that proposal into a reproducible profile. It stores immutable profile files, an epoch ledger, a champion pointer, element-usage statistics, and rejection experiences, so a change can be replayed and audited later. After initialization, the profile is rendered into maker context, and its identity records exactly which harness components produced the trajectory.

5.2 Loop Evolution

Once the proposer has seen both the attributed evidence A_t and the structured rejection memory $\mathcal{B}_t$, it distills them into a semantic diagnosis, and that diagnosis in turn constrains the mutation operator to a width-one change:

$$g_t = \Pi_{H_t^{\text{out}}}(A_t, \mathcal{B}_t), \qquad \widetilde{H}_{t+1}^{\text{in}} = \text{Mutate}_{w=1}\big(H_t^{\text{in}}, g_t\big). \tag{7}$$

The resulting candidate is then handed back to the inner loop, which performs the matched replay of Section 4.2 against the parent harness under the same cases and conditions that created the original champion. After the inner loop's protocol and rubric checks complete, the epoch can be recorded and the champion

pointer can change:

$$\text{PairedReplay}\left(H_t^{\text{in}}, \widetilde{H}_{t+1}^{\text{in}}; \mathcal{C}\right) \xrightarrow{\text{protocol + rubric checks}} H_{t+1}^{\text{in}} \in \left\{H_t^{\text{in}}, \widetilde{H}_{t+1}^{\text{in}}\right\}. \tag{8}$$

This width-one restriction keeps the causal interpretation local, since one epoch changes one harness dimension or one managed element category at a time. The outer epoch starts only after this inner decision is final, and its role is to react to the finished outcome, attributing it to the reusable elements the proposer drew on and updating those elements' evidence in the outer library; an element is credited precisely when its downstream inner candidate is admitted. Each element e of the library has the same bounded action space,

$$\mathcal{A}(e) = \{\text{add}, \text{delete}, \text{revise}, \text{merge}, \text{keep}\}, \tag{9}$$

and an outer epoch applies exactly one action to one element category, mirroring the width-one restriction of the inner loop. In the current implementation, this outer search is restricted to element-library management: it reshapes the reusable hub that the next inner proposal inherits, while the already-admitted inner game-facing harness is not changed by the outer update.

5.3 Verification

Unlike the inner-loop verification, the outer loop's evolvable components are managed as independent elements of a dynamic harness hub, so the hub judges them through usage, admission, and failure signals accumulated during the evolution process.[1] Each epoch updates how active elements behaved and leaves behind a record that later proposals can learn from. For an element e, the engine maintains a usage counter $u_t(e)$ and an admission counter $s_t(e)$, and it derives four signals that the hub uses to judge e:

$$\rho_t(e) = \frac{u_t(e)}{\max(1, \sum_{e'} u_t(e'))}, \quad \beta_t(e) = \frac{s_t(e)}{\max(1, u_t(e))}, \quad \beta_{t,c}(e) = \frac{s_{t,c}(e)}{\max(1, u_{t,c}(e))},$$

$$\mathcal{C}_t(e) = \{c \in \mathcal{C}_{\text{ho}} : u_{t,c}(e) > 0\}, \quad v_t(e) = \text{Var}_{c \in \mathcal{C}_t(e)}\big(\beta_{t,c}(e)\big), \quad \phi_t(e) = \mathbf{1}[h_t(e) > 0],$$

where $u_{t,c}(e)$ and $s_{t,c}(e)$ are the corresponding per-case usage and admission counts. Thus $\rho_t(e)$ is the usage rate of e among all loaded elements, $\beta_t(e)$ is its acceptance rate, and $v_t(e)$ is the variance of its observed per-case acceptance rates over $\mathcal{C}_t(e)$; we set $v_t(e) = 0$ when fewer than two cases have observations. Finally, $h_t(e)$ counts the episodes that loaded e and then failed a hard rubric, so the hard flag $\phi_t(e)$ records whether e was ever implicated in a legality failure. Later proposals can therefore rank elements by exposure, acceptance, and the stability of that acceptance, while the flag guards the legality gate regardless of soft scores. Once enough evidence has accumulated, usage-driven mutation can remove or rewrite well-tested low-accuracy elements, merge similar elements, or add derived variants when a category still has room. The updated counters are persisted in `element_stats.json`, and newly composed entries are appended to `element_catalog_extensions.json`; consequently, the library remains adaptive while the evaluator and benchmark rubric stay fixed.

6 Experiments

Having specified what each loop changes and how a candidate is admitted, we evaluate whether the resulting framework improves executable game artifacts across engines, benchmarks, and backbones. We then test whether the same mechanism transfers to broader agent tasks and whether its gains can be attributed to the evolved harness rather than to additional computation alone.

[1] We do not adopt the same A/B verification as used in the inner loop for two reasons: (1) The A/B testing for the inner loop harness only requires evaluating the game artifact, whereas the A/B testing for the harness of the outer loop requires measuring its *proposal capability*. An unbiased estimation of this capability requires multiple proposals across held-out samples, incurring prohibitive overhead. (2) Unlike inner-loop evolution, all evolvable elements in outer-loop evolution are designed and loaded via progressive disclosure, allowing us to naturally track their usage metadata.

6.1 Experiment Settings

Benchmarks. We evaluate the system on four game-oriented benchmarks (GameCraft-Bench, GameDev-Bench, V-GameGym, VeriGame) and two general agent benchmarks (Terminal-Bench, τ^2-Bench), covering both engine-based game generation and broader agent tasks.

GameCraft-Bench. GameCraft-Bench [32] asks the agent to produce runnable Godot games: 140 tasks spanning 15 game families, such as platformers, strategy, and roguelikes. Evaluation replays recorded input traces in a Godot runtime and scores hidden rubrics spanning mechanics, depth, visuals, and art.

GameDev-Bench. GameDev-Bench [5] contains 132 multi-file Godot development tasks derived from web and video tutorials, covering gameplay logic, 2D/3D graphics, and user-interface work. Tasks combine code with multimodal assets such as shaders, sprites, and animations, and their average solution requires more than three times the lines of code and file changes of typical software-engineering benchmarks; every candidate must remain importable and pass deterministic execution-based checks.

V-GameGym. V-GameGym [80] contains 2,219 Pygame game samples across 100 thematic clusters, each refactored over a fixed recording horizon so that code, screenshots, and video are all scored at once. The agent must therefore produce visible gameplay that goes beyond a static code patch.

VeriGame. VeriGame [15] evaluates web games through a verifier chain with runtime state injection, covering 100 games across seven genres and exposing inspectable gameplay variables. We add two rubrics of our own, Visual Usability (VU) and Intent Alignment (IA), alongside the benchmark's Build Health metric.

Terminal-Bench. Terminal-Bench [37] evaluates terminal workflows in a containerized official runner: 89 curated tasks spanning domains such as software engineering, system administration, and machine learning. Success requires completing tasks inside a constrained shell environment, which exercises tool use, command sequencing, and recovery under a different modality from game development.

τ^2-Bench. We use the official τ^2-Bench v1.0.1 simulator and evaluation protocol [3, 71], reporting Airline, Retail, Telecom, and Banking as in Table 5. The benchmark places the agent and a simulated user in a shared customer-service environment, so success depends on long-horizon communication, tool use, and policy compliance rather than on a single terminal action.

Implementation Details. We employ OpenGame [18] as the agent harness backbone, the first open-source agentic framework for end-to-end game creation. The vanilla baseline and awesome-gamedev-agent-skills [2] (a fixed skill library) serve as comparisons in all experiments. For the Game-Evolver experiments on a given dataset, all tasks in that dataset are available for evolution; however, we do not use the dataset's own public evaluation benchmarks, relying instead on our self-constructed rubrics for validation and feedback to avoid data leakage. The maximum number of evolution epochs is set to 35, and in each epoch we randomly sample 10% of the tasks from the original dataset (held-out sets excluded). Inner-loop rubric scoring is measured on a fixed 20% held-out set.

6.2 Experiment Results

Results on Four Game-Generation Benchmarks. Table 2 reports GameCraft-Bench results across six backbones. Game-Evolver raises the overall score on every backbone, with the largest relative gains occurring on the three weakest baselines. Qwen3.6-27B, DeepSeek-V4-Flash, and GLM5.2 improve by 182.7%, 100.8%, and 96.3%, while Kimi-K2.7-Code, Claude-Sonnet-4.6, and GPT-5.5 improve by 21.9%, 19.1%, and 6.8%. Two further patterns are directly visible. ① The fixed Awesome-Skills library raises the overall score of the three weakest baselines but lowers that of the three strongest, showing that a fixed skill set is not uniformly beneficial across backbone strengths. ② Mechanics and Visuals improve on all six backbones, whereas some individual rubric values decline, most notably Kimi-K2.7-Code's Depth and GPT-5.5's Art; the latter falls from 32.94 to 23.40, or 29.0%. The table establishes these performance patterns but does not by itself identify the underlying failure types, which are examined through runtime cases in Section 7.

[2] https://github.com/gamedev-skills/awesome-gamedev-agent-skills

Table 2 Results on GameCraft-Bench [32]. Mechanics, Depth, Visuals, and Art correspond to the four official rubric categories. The best scores are in bold.

Model	Method	Overall	Mechanics	Depth	Visuals	Art
Kimi-K2.7-Code	Baseline	20.06	28.25	18.30	25.23	16.09
	Awesome-Skills	18.37	23.35	**19.16**	**28.90**	10.93
	Game-Evolver	**24.45**	**39.61**	13.46	26.21	**28.20**
Qwen3.6-27B	Baseline	4.23	3.43	1.50	4.46	7.21
	Awesome-Skills	8.42	2.24	8.92	4.90	12.09
	Game-Evolver	**11.96**	**8.38**	**14.02**	**9.95**	**12.30**
GLM5.2	Baseline	18.00	26.50	20.17	21.15	10.84
	Awesome-Skills	20.67	25.24	27.30	20.01	12.35
	Game-Evolver	**35.33**	**43.69**	**36.71**	**37.98**	**29.22**
DeepSeek-V4-Flash	Baseline	6.31	6.10	7.13	9.18	4.35
	Awesome-Skills	9.69	6.03	13.95	5.89	8.62
	Game-Evolver	**12.67**	**13.42**	**14.27**	**13.10**	**10.55**
GPT-5.5	Baseline	39.47	54.36	38.61	41.84	**32.94**
	Awesome-Skills	38.12	55.44	42.32	42.30	24.70
	Game-Evolver	**42.17**	**58.49**	**48.87**	**54.00**	23.40
Claude-Sonnet-4.6	Baseline	22.64	37.02	24.38	20.38	15.71
	Awesome-Skills	17.68	31.95	19.99	24.91	6.15
	Game-Evolver	**26.96**	**46.30**	**27.50**	**30.53**	**16.59**

Table 3 reports the results on GameDev-Bench. We have the following findings. ① 3D Graphics & Animation is the lowest baseline channel for GPT-5.5, GLM5.2, and Claude-Sonnet-4.6; for GPT-5.5 it is 21.21, compared with 77.24 in 2DGA. The largest channel-level changes also occur in 3DGA: Kimi-K2.7-Code rises from 3.03 to 15.15, a gain of 400%, and DeepSeek-V4-Flash from 12.12 to 48.48, a gain of 300%. ② Overall gains are largest on the two weakest baselines, with DeepSeek-V4-Flash improving by 300.1% and Kimi-K2.7-Code by 130.0%; GLM5.2 is an exception to a strict inverse ordering, gaining 36.9% despite having the second-highest baseline. Gameplay Logic rises on Kimi-K2.7-Code, Qwen3.6-27B, and DeepSeek-V4-Flash, but falls on GLM5.2, GPT-5.5, and Claude-Sonnet-4.6 by 14.6%, 13.9%, and 5.3%, respectively. ③ Awesome-Skills raises DeepSeek-V4-Flash from 12.31 to 48.05, a gain of 290.3%, but remains below Game-Evolver on all six backbones and lowers the GPT-5.5 and Claude-Sonnet-4.6 overall scores. Thus the fixed library transfers unevenly across models on these tutorial-derived tasks.

Table 4 covers V-GameGym and VeriGame. On V-GameGym, overall gains are universal and range from 4.1% to 28.9%. The largest relative Video increases occur on Qwen3.6-27B and Claude-Sonnet-4.6, whose scores rise by 107.9% and 36.6%, respectively; Code, by contrast, changes little or declines on three backbones. On VeriGame, every backbone gains at least 22.1% overall, and Build Health rises on all six, with Claude-Sonnet-4.6 climbing from 26.67 to 47.40, a gain of 77.7%. Visual Usability increases by 26.0% to 112.2%, while Intent Alignment changes less on average and drops slightly for Claude-Sonnet-4.6. These channel differences are consistent with a stronger effect on executable and visible behavior than on intent matching, although the table alone does not identify the mechanism. Awesome-Skills improves only GLM5.2 overall on V-GameGym and only Kimi-K2.7-Code and DeepSeek-V4-Flash overall on VeriGame, showing limited transfer of the fixed library across these runtimes.

Different Backbones. Across the four suites, GPT-5.5 is the strongest baseline on every suite. Its relative gains are 6.8% on GameCraft-Bench, 4.3% on GameDev-Bench, and 4.1% on V-GameGym, but 29.6% on VeriGame. GLM5.2 is never the strongest baseline, yet it gains 22.1% to 96.3% and overtakes GPT-5.5 on GameDev-Bench and V-GameGym. Claude-Sonnet-4.6 records the largest relative gain on VeriGame at 67.9%, despite ranking near the bottom there before evolution. Qwen3.6-27B and DeepSeek-V4-Flash obtain several of the largest relative gains, but their evolved absolute scores do not uniformly surpass the stronger backbones. Taken together, these results show that relative gain depends on both backbone and runtime; they do not support a single monotonic relation between baseline strength and improvement.

Generalization to broad agent tasks. Figure 4 and Table 5 show that Game-Evolver transfers to broad agent tasks beyond game generation. On Terminal-Bench 2.0, GLM5.2 improves from 62.1 to 69.4 and Qwen3.6-27B from 48.2 to 52.7; on τ^2-Bench v1.0.1, GLM5.2 rises from 77.1 to 82.1 and Qwen3.6-27B from 67.2 to

Table 3 Results on GameDev-Bench [5]. 2D Graphics & Animation (2DGA), 3D Graphics & Animation (3DGA), User Interface, and Gameplay Logic are the four sub-types.

Model	Method	Overall	2DGA	3DGA	User Interface	Gameplay Logic
Kimi-K2.7-Code	Baseline	3.00	2.76	3.03	4.11	2.44
	Awesome-Skills	3.60	2.76	9.09	1.37	**4.88**
	Game-Evolver	**6.90**	**6.20**	**15.15**	**6.85**	**4.88**
Qwen3.6-27B	Baseline	30.93	28.28	18.18	36.99	35.37
	Awesome-Skills	34.23	39.31	18.18	35.62	30.49
	Game-Evolver	**43.24**	**51.72**	**24.24**	**38.36**	**40.24**
GLM5.2	Baseline	53.75	53.79	42.42	53.42	**58.54**
	Awesome-Skills	60.66	77.93	30.30	57.53	45.12
	Game-Evolver	**73.57**	**85.52**	**75.76**	**75.34**	50.00
DeepSeek-V4-Flash	Baseline	12.31	7.59	12.12	10.96	21.95
	Awesome-Skills	48.05	41.38	**54.55**	**56.16**	50.00
	Game-Evolver	**49.25**	**46.21**	48.48	52.05	**52.44**
GPT-5.5	Baseline	55.56	77.24	21.21	41.10	**43.90**
	Awesome-Skills	50.75	62.76	39.39	**46.58**	37.80
	Game-Evolver	**57.96**	**80.00**	**45.45**	42.47	37.80
Claude-Sonnet-4.6	Baseline	50.75	**54.48**	18.18	63.01	**46.34**
	Awesome-Skills	41.44	41.38	21.21	54.79	37.80
	Game-Evolver	**52.85**	51.72	**36.36**	**72.60**	43.90

Table 4 Results on V-GameGym [80] and VeriGame [15]. Code, Screenshot, and Video are the official rubrics of V-GameGym; Build Health (BH) is the hard channel of VeriGame, and Visual Usability (VU) and Intent Alignment (IA) are two additional VeriGame rubrics defined by ourselves. Their detailed definitions can be found in Section A.

Model	Method	V-GameGym				VeriGame			
		Overall	Code	Screenshot	Video	Overall	BH	VU	IA
Kimi-K2.7-Code	Baseline	36.59	78.96	17.01	13.79	40.90	63.20	23.50	40.20
	Awesome-Skills	31.31	63.91	15.36	14.67	43.12	65.46	22.76	41.14
	Game-Evolver	**39.69**	**82.42**	**20.46**	**16.18**	**55.96**	**73.09**	**49.87**	**48.96**
Qwen3.6-27B	Baseline	30.03	73.00	11.30	5.79	35.38	52.10	27.86	36.57
	Awesome-Skills	29.93	73.50	9.49	6.80	35.00	52.76	30.92	35.08
	Game-Evolver	**34.20**	**74.63**	**15.94**	**12.04**	**47.65**	**65.13**	**43.91**	**38.17**
GLM5.2	Baseline	37.60	72.78	17.89	22.13	46.19	60.56	37.19	42.06
	Awesome-Skills	42.46	82.42	23.16	21.80	41.22	70.87	35.70	30.83
	Game-Evolver	**48.45**	**90.79**	**29.43**	**25.11**	**56.39**	**73.75**	**50.63**	**48.18**
DeepSeek-V4-Flash	Baseline	37.90	**76.46**	18.37	18.86	21.87	42.50	11.91	13.65
	Awesome-Skills	33.06	72.16	12.60	14.43	23.21	43.96	8.49	7.17
	Game-Evolver	**41.60**	76.38	**27.12**	**21.30**	**30.35**	**45.74**	**24.26**	**20.13**
GPT-5.5	Baseline	43.87	**85.86**	23.53	22.21	50.50	56.25	44.50	50.75
	Awesome-Skills	40.49	80.89	16.10	24.47	48.54	56.90	40.44	48.94
	Game-Evolver	**45.67**	84.05	**24.52**	**28.43**	**65.45**	**72.06**	**56.08**	**59.56**
Claude-Sonnet-4.6	Baseline	28.59	**63.12**	11.81	10.84	22.93	26.67	19.47	**22.67**
	Awesome-Skills	28.47	60.59	14.03	10.78	18.47	20.00	17.07	18.33
	Game-Evolver	**30.66**	62.12	**15.05**	**14.81**	**38.50**	**47.40**	**29.54**	22.16

70.7. The resulting gains range from 3.5 to 7.3 points. On τ^2-Bench, both models gain most in Banking, which also has their lowest baseline class scores: GLM5.2 rises from 36.1 to 47.4 and Qwen3.6-27B from 17.5 to 26.8. Telecom stays flat or rises slightly, while Airline drops for both evolved models. The evolved GLM5.2 reaches 82.1 and overtakes the GPT-5.5 baseline at 78.9, reproducing the ranking reshuffle seen on GameDev-Bench and V-GameGym. These results establish transfer, while the figure and table alone do not explain why the gains differ across benchmark families.

7 Analysis

The aggregate results establish that Game-Evolver improves the evaluated baselines. We next examine where those gains occur, how harness proposals are accepted over time, and which loop components account for the improvement and its cost.

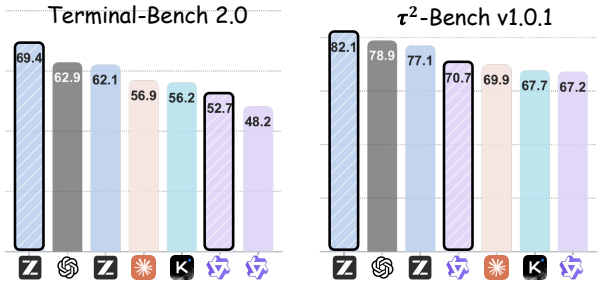

Figure 4 Results on Terminal-Bench and τ^2-Bench. Bars with black contours are those with Game-Evolver.

Table 5 Per-class results on τ^2-Bench. $*$: results contain recurring infrastructure issues.

Model	Airline	Retail	Telecom	Banking	Avg
Kimi-K2.7-Code	78.0	81.6	93.0	16.5	67.7
Qwen3.6-27B	86.0	76.3*	92.1	17.5	67.2
GLM5.2	92.0	83.3	99.1	36.1	77.1
Claude4.6-sonnet	68.0	89.5	80.7	35.1	69.9
GPT-5.5	94.0	88.6	98.2	37.1	78.9
Qwen3.6-27B	**86.0**	76.3*	**92.1**	17.5	67.2
+Game-Evolver	84.0	**80.7**	**92.1**	**26.8**	**70.7**
GLM5.2	**92.0**	83.3	99.1	36.0	77.1
+Game-Evolver	88.0	**91.2**	**100.0**	**47.4**	**82.1**

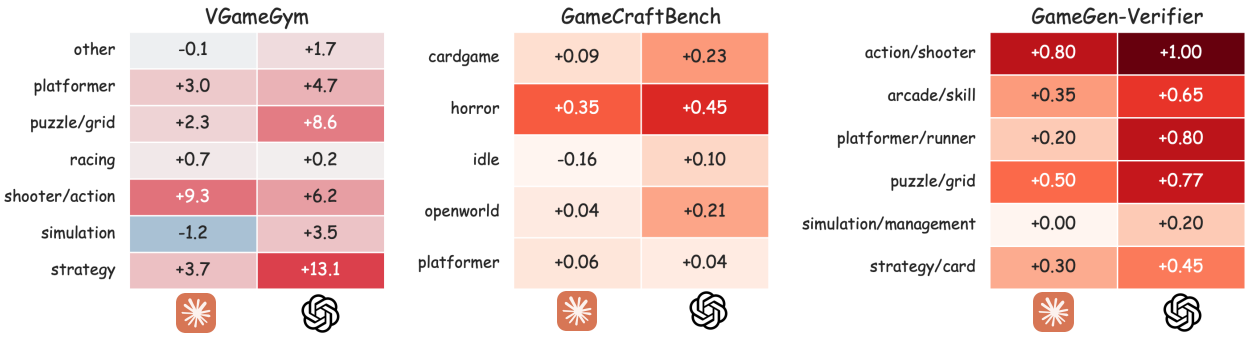

Figure 5 By-type improvement of Game-Evolver compared with the baseline on V-GameGym, GameCraft-Bench, and VeriGame (labeled GameGen-Verifier in the source plot). The left and right columns in each panel correspond to Claude-Sonnet-4.6 and GPT-5.5, respectively.

7.1 By-type Analysis

Figure 5 decomposes the Game-Evolver gain by game type across V-GameGym, GameCraft-Bench, and VeriGame. Each cell is the delta over the baseline for one type under one backbone. The gain is concentrated: a minority of types absorbs most of the improvement in every panel. On V-GameGym, strategy gains +13.1 on GPT-5.5 and shooter/action +9.3 on Claude, while racing changes by only +0.7/+0.2. On GameCraft-Bench, horror leads at +0.35/+0.45, whereas platformer is nearly flat at +0.06/+0.04. On VeriGame, action/shooter leads at +0.80/+1.00, platformer/runner is also strong on GPT-5.5 at +0.80, and simulation/management changes by +0.00/+0.20. These values establish substantial type-level heterogeneity across all three suites.

One interpretation of this heterogeneity is that bounded replay provides stronger evidence for some task types than for others. The high-gain groups in the plot, including shooter/action, strategy, horror, and platformer/runner, expose observable state changes during play, whereas idle and simulation/management show smaller changes in the displayed comparisons. This interpretation is also consistent with the aggregate channel results: averaged across the six backbones, Video has the largest relative increase among the three V-GameGym channels, while Intent Alignment has the smallest relative increase among the three added VeriGame channels. The plots do not isolate replay observability as the cause, however, because task difficulty, baseline quality, and rubric composition also vary across types.

The two backbone columns add a second descriptive pattern. In fifteen of eighteen cells, GPT-5.5's per-type gain exceeds Claude's. All three regressions, -0.1, -1.2, and -0.16, occur in the Claude column, and the only +1.00 cell, on VeriGame action/shooter, occurs in the GPT-5.5 column. Thus GPT-5.5 has the larger absolute per-type change in this two-backbone slice, even though the aggregate tables show the largest relative overall gains on several weaker baselines. The two observations use different denominators and levels of aggregation, so they are complementary rather than contradictory; neither figure alone identifies the model-side mechanism behind the difference.

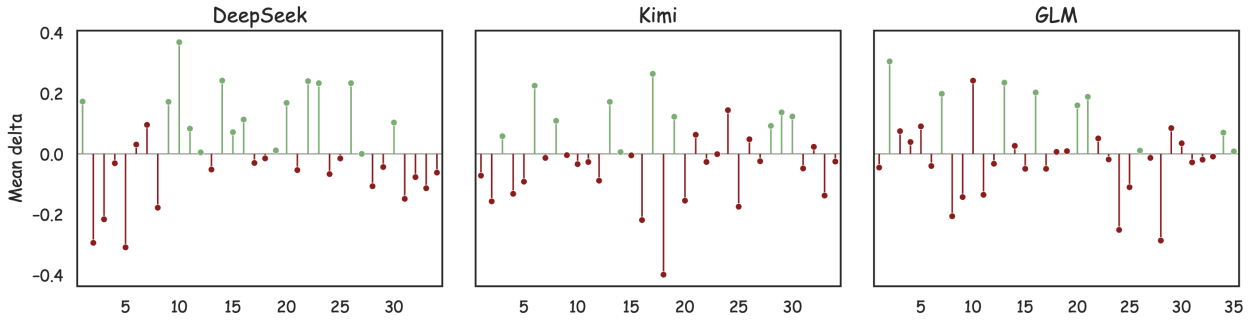

Figure 6 Per-epoch change of the inner loop's self-built dual-rubric scores for harness proposed by the HPA, for the DeepSeek, Kimi, and GLM backbones. Green marks accepted candidates and red marks rejected candidates.

7.2 Scoring Dynamics

Figure 6 reports, per epoch, the change in the inner loop's mean dual-rubric score for each harness modification proposed by the HPA and whether the matched A/B test accepts or rejects it, for the DeepSeek, Kimi, and GLM backbones. Every accepted candidate shown in green has a nonnegative mean change, while most rejected candidates shown in red have a negative change. The figure also contains rejected candidates with positive mean deltas. Under the admission rule, this can occur when the scalar mean improves but another required condition, such as a hard rubric, protocol check, or per-case non-regression constraint, fails. The plot is therefore consistent with a multi-condition admission gate rather than scalar hill climbing.

The three backbones differ in the visible distribution of outcomes. DeepSeek contains more green markers than Kimi or GLM and records several accepted gains above +0.2. Kimi contains the lowest point, approximately -0.4, while GLM's final epochs lie mostly near zero and include only two accepted changes after epoch 30. These are descriptive properties of the plotted proposals. The figure does not by itself establish whether they arise from proposal quality, saturation, backbone behavior, or sampling variation.

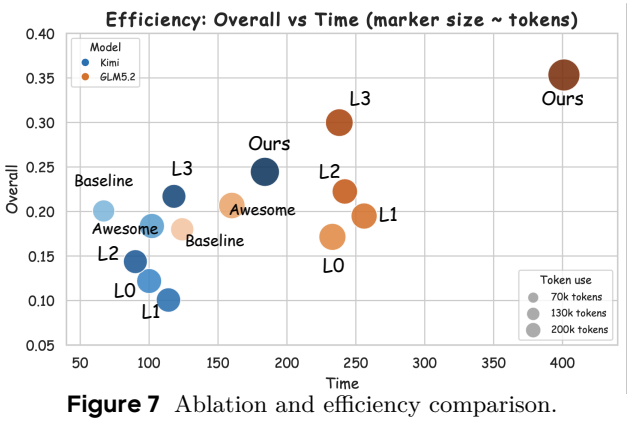

Figure 7 Ablation and efficiency comparison.

7.3 Ablation and Cost Analysis

Figure 7 compares the ablation levels L0 to L4 on GameCraft-Bench, with overall score on the y-axis, time cost on the x-axis, and token consumption encoded as point size; Baseline and Awesome-Skills are reference points. L0 to L2 occupy the lower-score region for both backbones. L0 runs the model alone, L1 freezes the OpenGame harness, and L2 evolves only context-related components; their ordering is not monotonic, and Baseline is at or above some of these points. Within this ablation, context-only evolution is therefore not uniformly beneficial. L3, which also allows tools, policies, and modules to change, reaches about 0.22 for Kimi and 0.30 for GLM5.2, above the corresponding L0–L2 points. L4 reaches the highest plotted score for both backbones, about 0.25 and 0.35. Relative to L3, its time coordinate increases from roughly 120 to 185 for Kimi and from 240 to 400 for GLM5.2, or approximately 1.5–1.7×. Because L4 adds the outer loop to L3, the comparison attributes the additional plotted score and cost to that component under this experimental setup. Whether the reusable harness amortizes the cost on future tasks remains a hypothesis rather than a quantity measured in the figure.

7.4 Case Study

Evolved harness for GOA agents. Table 6 lists eight admitted harness changes and their observed mean-rubric-score differences. All rubric scores increases, from +0.058 to +0.258. The two largest increases are associated

Old harness	New harness	Change Description	Mean Rubric Score
Validated Godot resource imports before making gameplay edits.	Discovered and validated the runnable project entry point, including the project manifest, main scene, and entry scripts, before editing.	Shifted preprocessing from general import validation to identifying the executable path used by subsequent playtests and verification.	0.242 → 0.467 (+0.225)
Validated declared dependencies, add-ons, and referenced resources before gameplay edits.	Ran Godot import validation before editing and recorded scene, script, and resource parsing failures.	Replaced dependency-level validation with engine-level import and parsing checks.	0.198 → 0.418 (+0.220)
Validated Godot resource imports and scene parsing before gameplay edits.	Validated declared dependencies, add-ons, imported resources, and referenced files before gameplay edits.	Broadened pre-edit validation from engine import errors to missing or inconsistent project dependencies.	0.193 → 0.328 (+0.135)
Blocked candidate acceptance until at least one deep runtime probe succeeded.	Required explicit planning, editing, and verification phases before candidate acceptance.	Replaced a single runtime gate with a structured edit-to-verification handoff requiring a separate verification phase.	0.307 → 0.365 (+0.058)
Required an explicit transition from editing to verification before accepting a candidate.	Blocked candidate acceptance until at least one deep runtime probe completed successfully.	Replaced a procedural handoff requirement with an evidence-based acceptance gate tied to successful runtime execution.	0.353 → 0.612 (+0.258)
Probed whether the project had a valid executable entry point.	Audited deterministic demonstration replays for observable gameplay evidence.	Shifted validation from confirming that the game could launch to confirming that demonstrations exercised meaningful gameplay states and interactions.	0.153 → 0.303 (+0.150)
Created a checkpoint before editing and rolled back changes after a hard regression.	Ran deep runtime probes before diagnosing problems and proposing gameplay patches.	Changed the workflow from reactive rollback to evidence-first diagnosis, so patches were informed by runtime behavior before implementation.	0.350 → 0.438 (+0.088)
Added engine-specific hints inferred from the workspace structure to the agent context.	Added summaries of the four most recent patches and their outcomes to the agent context.	Replaced static engine guidance with short-term edit history intended to prevent repeated unsuccessful modifications.	0.152 → 0.327 (+0.175)

Table 6 Accepted harness evolution cases and their score change.

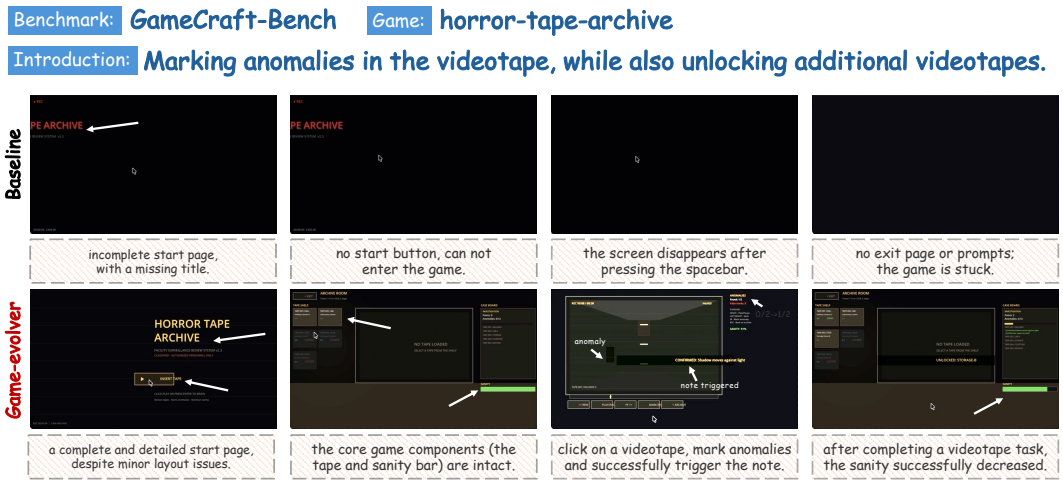

Figure 8 Case #1: horror-tape-archive from GameCraft-Bench. The images above are screenshots of games generated by the baseline, and it scored 0.5763; below are that generated by Game-Evolver, and it scored 0.7702.

with replacing a phase-handoff requirement by a successful deep runtime probe (+0.258) and replacing general import validation by executable-entry-point discovery (+0.225); engine-level import validation is close behind at +0.220. The table also contains dependency validation, replay auditing, checkpoint replacement, and context-history changes, so the admitted set is broader than runtime probing alone. Two rows reverse the same pair of rules: the phase-handoff direction records +0.058, whereas the deep-runtime-probe direction records +0.258. This difference is consistent with the evidence gate being more useful in these logged cases, but the rows start from different mean scores and are not presented as a controlled head-to-head experiment. The supported conclusion is therefore narrower: among the listed cases, the largest observed gains are attached to changes that expose executable runtime evidence.

Game Property. Figure 8 shows a case from GameCraft-Bench and the interaction gap behind the result from baseline and Game-Evolver. In the displayed baseline trace, the title page is incomplete, no start action is available, and later frames become blank or stuck after input. The evolved trace shows a complete start flow, the tape and sanity interfaces, an anomaly-marking response, and a subsequent sanity decrease. The screenshots therefore document a change from a broken interaction path to the requested visible state transitions. They are consistent with the framework's runtime-oriented verification, although the figure alone does not identify which internal probe produced the repair.

Another case from GameDev-Bench is shown in Figure 9. The baseline workspace shows depth fog, volumetric fog, and LUT correction disabled in the shared `WorldEnvironment`, whereas the evolved workspace shows all three enabled together with the required renderer setting. The scene can therefore be structurally present while still missing the requested visual configuration. This before/after pair directly documents correction of semantic incompleteness that a file-existence check would not detect; it does not, by itself, reveal which verification branch selected the edit. Additional cases from GameCraft-Bench, GameDev-Bench, V-GameGym, and VeriGame are collected in Section B.

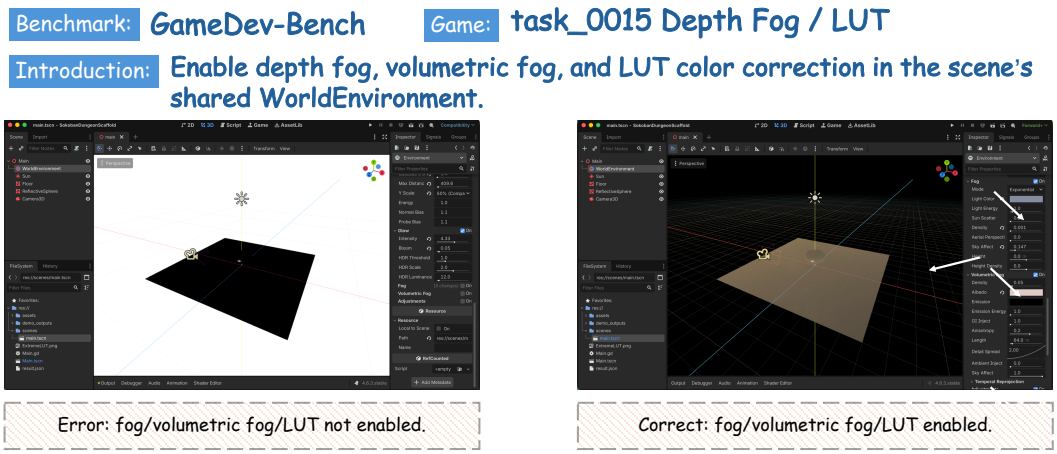

Figure 9 Case #2: task_0015 Depth Fog / LUT from GameDev-Bench. The left image is the Godot workspace of the baseline (score: 0.0); the right image shows Game-Evolver (score: 1.0).

8 Conclusion

We presented Game-Evolver, a dual-loop framework that couples game-artifact execution with harness evolution. A game orchestration agent (GOA) drives the inner loop, turning a specification into a runnable game under the current champion harness and collecting the evidence needed for judgment. The inner loop then A/B-tests the game runtime made with the new harness, and the candidate is admitted only when matched replay passes dual-rubric verification. A harness proposal agent (HPA) drives the outer loop, distilling recurring failure families into a bounded harness element change. Our design guarantees that both the game-making harness and the harness-proposal loop can be continually improved after deployment, which boosts the long-horizon evolution of agent capability but agnostic of game or infrastructure types.

Across four game benchmarks and six backbones, Game-Evolver improved every baseline, lifting the weakest models by several-fold and the strongest by up to 30%, and it transferred to general agent tasks on Terminal-Bench and τ^2-Bench. The type-level analysis sharpens the boundary of the approach: evolution acts where running the artifact can expose a gap, repairing what runtime verification reveals, with its reach ending where a short play-through runs out of evidence. By making gains attributable and auditable, Game-Evolver positions the harness as a first-class object of study, elevating it beyond a fixed scaffold. Two directions follow. More adaptive dynamic rubrics could let the verification criteria track task evidence more closely while the admission rule stays fixed, and the outer loop's own evolution could progressively refine the reusable element library that shapes future harness proposals, so that the loop that proposes changes is itself improved by accumulated evidence.

References

[1] Saaket Agashe, Kyle Wong, Vincent Tu, Jiachen Yang, Ang Li, and Xin Eric Wang. Agent s2: A compositional generalist-specialist framework for computer use agents. arXiv preprint arXiv:2504.00906, 2025.

[2] Bowen Baker, Ilge Akkaya, Peter Zhokhov, Joost Huizinga, Jie Tang, Adrien Ecoffet, Brandon Houghton, Raul Sampedro, and Jeff Clune. Video pretraining: Learning to act by watching unlabeled online videos. In Advances in Neural Information Processing Systems, 2022.

[3] Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. tau^2-bench: Evaluating conversational agents in a dual-control environment. arXiv preprint arXiv:2506.07982, 2025.

[4] Christopher Berner, Greg Brockman, Brooke Chan, Vicki Cheung, Przemysław Dębiak, Christy Dennison, David Farhi, Quirin Fischer, Shariq Hashme, Chris Hesse, Rafal Józefowicz, Scott Gray, Catherine Olsson, Jakub Pachocki, Michael Petrov, Henrique P. de Oliveira Pinto, Jonathan Raiman, Tim Salimans, Jeremy Schlatter, Jonas Schneider, Szymon Sidor, Ilya Sutskever, Jie Tang, Filip Wolski, and Susan Zhang. Dota 2 with large scale deep reinforcement learning. arXiv preprint arXiv:1912.06680, 2019.

[5] Wayne Chi, Yixiong Fang, Arnav Yayavaram, Siddharth Yayavaram, Seth Karten, Qiuhong Anna Wei, Runkun Chen, Alexander Wang, Valerie Chen, Ameet Talwalkar, et al. Gamedevbench: Evaluating agentic capabilities through game development. arXiv preprint arXiv:2602.11103, 2026.

[6] Paul F Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. Advances in neural information processing systems, 30, 2017.

[7] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. In International Conference on Machine Learning, 2024.

[8] Linxi Fan, Guanzhi Wang, Yunfan Jiang, Ajay Mandlekar, Yuncong Yang, Haoyi Zhu, Andrew Tang, De-An Huang, Yuke Zhu, and Anima Anandkumar. Minedojo: Building open-ended embodied agents with internet-scale knowledge. In Advances in Neural Information Processing Systems, 2022.

[9] Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. Promptbreeder: Self-referential self-improvement via prompt evolution. In International Conference on Machine Learning, 2024.

[10] Qingyan Guo, Rui Wang, Junliang Guo, Bei Li, Kaitao Song, Xu Tan, Guoqing Liu, Jiang Bian, and Yujiu Yang. Connecting large language models with evolutionary algorithms yields powerful prompt optimizers. In International Conference on Learning Representations, 2024.

[11] Izzeddin Gur, Hiroki Furuta, Austin V. Huang, Mustafa Safdari, Yutaka Matsuo, Douglas Eck, and Aleksandra Faust. A real-world webagent with planning, long context understanding, and program synthesis. In International Conference on Learning Representations, 2024.

[12] Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. Metagpt: Meta programming for a multi-agent collaborative framework. In International Conference on Learning Representations, 2024.

[13] Wenyi Hong, Weihan Wang, Qingsong Lv, Jiazheng Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxiao Dong, Ming Ding, et al. Cogagent: A visual language model for gui agents. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2024.

[14] Yixu Huang, Bo Li, Na Li, Zhe Wang, Kaijie Chen, Haonan Ge, Qingyi Si, Yuanzhe Shen, Ruihan Yang, Guangjing Wang, and Hongcheng Guo. Gui agents for continual game generation. arXiv preprint arXiv:2605.28258, 2026.

[15] Chaobo Jia, Ruipeng Wan, Ting Sun, Weihao Tan, Borui Wan, Yuxuan Tong, Guangming Sheng, and Hong Xu. Gamegen-verifier: Parallel keypoint-based verification for llm-generated games via runtime state injection. arXiv preprint arXiv:2605.07442, 2026.

[16] Shufan Jiang, Chios Chen, and Zhiyang Chen. Gbqa: A game benchmark for evaluating llms as quality assurance engineers. arXiv preprint arXiv:2604.02648, 2026.

[17] Xue Jiang, Yihong Dong, Mengyang Liu, Deng Hongyi, Tian Wang, Yongding Tao, Zhi Jin, Wenpin Jiao, and Ge Li. Coderl+: Improving code generation via reinforcement with execution semantics alignment. In Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 3608–3622, 2026.

[18] Yilei Jiang, Jinyuan Hu, Qianyin Xiao, Yaozhi Zheng, Ruize Ma, Kaituo Feng, Jiaming Han, Tianshuo Peng, Kaixuan Fan, Manyuan Zhang, and Xiangyu Yue. Opengame: Open agentic coding for games. arXiv preprint arXiv:2604.18394, 2026.

[19] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? In International Conference on Learning Representations, 2024.

[20] Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-r1: Training llms to reason and leverage search engines with reinforcement learning. arXiv preprint arXiv:2503.09516, 2025.

[21] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. Dspy: Compiling declarative language model calls into self-improving pipelines. arXiv preprint arXiv:2310.03714, 2023.

[22] Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Russ Salakhutdinov, and Daniel Fried. Visualwebarena: Evaluating multimodal agents on realistic visual web tasks. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 881–905, 2024.

[23] Hanyu Lai, Xiao Liu, Yanxiao Zhao, Han Xu, Hanchen Zhang, Bohao Jing, Yanyu Ren, Shuntian Yao, Yuxiao Dong, and Jie Tang. Computerrl: Scaling end-to-end online reinforcement learning for computer use agents. In International Conference on Learning Representations, volume 2026, pages 23553–23591, 2026.

[24] Changxin Lao, Fei Pan, Guozhuang Ma, Han Li, Huihuang Lin, Jijun Shi, Kangzhi Zhao, Kun Gai, Mo Zhou, Qinqin Zhou, et al. Agentx: Towards agent-driven self-iteration of industrial recommender systems. arXiv preprint arXiv:2606.26859, 2026.

[25] Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven C. H. Hoi. Coderl: Mastering code generation through pretrained models and deep reinforcement learning. In Advances in Neural Information Processing Systems, 2022.

[26] Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses. arXiv preprint arXiv:2603.28052, 2026.

[27] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. Camel: Communicative agents for mind exploration of large language model society. In Advances in Neural Information Processing Systems, 2023.

[28] Xiaoxi Li, Wenxiang Jiao, Jiarui Jin, Guanting Dong, Jiajie Jin, Yinuo Wang, Hao Wang, Yutao Zhu, Ji-Rong Wen, Yuan Lu, et al. Deepagent: A general reasoning agent with scalable toolsets. In Proceedings of the ACM Web Conference 2026, pages 2219–2230, 2026.

[29] Jiahang Lin, Shichun Liu, Chengjun Pan, Lizhi Lin, Shihan Dou, Xuanjing Huang, Hang Yan, Zhenhua Han, and Tao Gui. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses. arXiv preprint arXiv:2604.25850, 2026.

[30] Minhua Lin, Juncheng Wu, Zijun Wang, Zhan Shi, Yisi Sang, Bing He, Zewen Liu, Tianxin Wei, Zongyu Wu, Zhiwei Zhang, Dakuo Wang, Xiang Zhang, Benoit Dumoulin, Cihang Xie, Yuyin Zhou, Suhang Wang, and Hanqing Lu. Harness updating is not harness benefit: Disentangling evolution capabilities in self-evolving llm agents. arXiv preprint arXiv:2605.30621, 2026.

[31] Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding r1-zero-like training: A critical perspective. arXiv preprint arXiv:2503.20783, 2025.

[32] Tongxu Luo, Rongsheng Wang, Jiaxi Bi, Chenming Xu, Zhengyang Tang, Jianlong Chen, Juhao Liang, Ke Ji, Shuqi Guo, Yuhao Du, et al. Gamecraft-bench: Can agents build playable games end-to-end in a real game engine? arXiv preprint arXiv:2606.17861, 2026.

[33] Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Eureka: Human-level reward design via coding large language models. In International Conference on Learning Representations, 2024.

[34] Ziyu Ma, Shidong Yang, Yuxiang Ji, Xucong Wang, Yong Wang, Yiming Hu, Tongwen Huang, and Xiangxiang Chu. Skillclaw: Let skills evolve collectively with agentic evolver, 2026. URL https://arxiv. org/abs/2604.08377.

[35] Ziyu Ma, Shidong Yang, Yuxiang Ji, Xucong Wang, Yong Wang, Yiming Hu, Tongwen Huang, and Xiangxiang Chu. Skillclaw: Let skills evolve collectively with agentic evolver. arXiv preprint arXiv:2604.08377, 2026.

[36] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. Advances in neural information processing systems, 36:46534–46594, 2023.

[37] Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. arXiv preprint arXiv:2601.11868, 2026.

[38] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. nature, 518(7540):529–533, 2015.

[39] Xuying Ning, Katherine Tieu, Dongqi Fu, Tianxin Wei, Zihao Li, Yuanchen Bei, Jiaru Zou, Mengting Ai, Zhining Liu, Ting-Wei Li, et al. Code as agent harness. arXiv preprint arXiv:2605.18747, 2026.

[40] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. Alphaevolve: A coding agent for scientific and algorithmic discovery. arXiv preprint arXiv:2506.13131, 2025.

[41] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. Advances in neural information processing systems, 35:27730–27744, 2022.

[42] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In ACM Symposium on User Interface Software and Technology, 2023.

[43] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. Chatdev: Communicative agents for software development. In Proceedings of the Annual Meeting of the Association for Computational Linguistics, 2024.

[44] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. Advances in neural information processing systems, 36:53728–53741, 2023.

[45] Christopher Rawles, Alice Li, Daniel Rodriguez, Oriana Riva, and Timothy Lillicrap. Android in the wild: A large-scale dataset for android device control. In Advances in Neural Information Processing Systems, 2024.

[46] Esteban Real, Chen Liang, David R. So, and Quoc V. Le. Automl-zero: Evolving machine learning algorithms from scratch. In International Conference on Machine Learning, pages 8007–8019. PMLR, 2020.

[47] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. Nature, 625: 468–475, 2024.

[48] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347, 2017.

[49] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In Advances in Neural Information Processing Systems, 2023.

[50] Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Cote, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. Alfworld: Aligning text and embodied environments for interactive learning. In International Conference on Learning Representations, 2021.

[51] David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel, and Demis Hassabis. Mastering the game of go with deep neural networks and tree search. Nature, 529: 484–489, 2016.

[52] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, Yutian Chen, Timothy Lillicrap, Fan Hui, Laurent Sifre, George van den Driessche, Thore Graepel, and Demis Hassabis. Mastering the game of go without human knowledge. Nature, 550:354–359, 2017.

[53] Richard S Sutton, Andrew G Barto, and Andrew Barto. Reinforcement learning: An introduction, volume 1. MIT press Cambridge, 1998.

[54] Weihao Tan, Wentao Zhang, Xinrun Xu, Haochong Xia, Ziluo Ding, Boyu Li, Bohan Zhou, Junpeng Yue, Jiechuan Jiang, Yewen Li, et al. Cradle: Empowering foundation agents towards general computer control. arXiv preprint arXiv:2403.03186, 2024.

[55] Oriol Vinyals, Igor Babuschkin, Wojciech M. Czarnecki, Michaël Mathieu, Andrew Dudzik, Junyoung Chung, David H. Choi, Richard Powell, Timo Ewalds, Petko Georgiev, Junhyuk Oh, Dan Horgan, Manuel Kroiss, Ivo Danihelka, Aja Huang, Laurent Sifre, Trevor Cai, John P. Agapiou, Max Jaderberg, Alexander S. Vezhnevets, Rémi Leblond, Tobias Pohlen, Valentin Dalibard, David Budden, Yury Sulsky, James Molloy, Tom L. Paine, Caglar Gulcehre, Ziyu Wang, Tobias Pfaff, Yuhuai Wu, Roman Ring, Dani Yogatama, Dario Wünsch, Katrina McKinney, Oliver Smith, Tom Schaul, Timothy Lillicrap, Koray Kavukcuoglu, Demis Hassabis, Chris Apps, and David Silver. Grandmaster level in starcraft ii using multi-agent reinforcement learning. Nature, 575:350–354, 2019.

[56] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. Transactions on Machine Learning Research, 2024.

[57] Junyang Wang, Haiyang Xu, Haitao Jia, Xi Zhang, Ming Yan, Weizhou Shen, Ji Zhang, Fei Huang, and Jitao Sang. Mobile-agent-v2: Mobile device operation assistant with effective navigation via multi-agent collaboration. In Advances in Neural Information Processing Systems, 2024.

[58] Peiyi Wang, Lei Li, Zhihong Shao, Runxin Xu, Damai Dai, Yifei Li, Deli Chen, Yu Wu, and Zhifang Sui. Math-shepherd: Verify and reinforce llms step-by-step without human annotations. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 9426–9439, 2024.

[59] Xucong Wang, Pengchao Han, and Lei Guo. Lightweight self-knowledge distillation with multi-source information fusion. arXiv preprint arXiv:2305.09183, 2023.

[60] Xucong Wang, Pengkun Wang, Shurui Zhang, Miao Fang, and Yang Wang. Multi-label self knowledge distillation. In Proceedings of the AAAI Conference on Artificial Intelligence, volume 39, pages 21330–21338, 2025.

[61] Xucong Wang, Ziyu Ma, Shidong Yang, Tongwen Huang, Pengkun Wang, Yong Wang, and Xiangxiang Chu. Role-agent: Bootstrapping llm agents via dual-role evolution. arXiv preprint arXiv:2606.10917, 2026.

[62] Xucong Wang, Zhe Zhao, Liheng Yu, Di Wu, Xiaofeng Cao, and Pengkun Wang. Didpo: Diff-in-diff policy optimization for coding agent training. arXiv preprint arXiv:2608.07147, 2026.

[63] Xuezhi Wang, Jason Wei, Dale Schuurmans, et al. Self-consistency improves chain of thought reasoning in language models. In International Conference on Learning Representations, 2023.

[64] Zihan Wang, Kangrui Wang, Qineng Wang, Pingyue Zhang, Linjie Li, Zhengyuan Yang, Xing Jin, Kefan Yu, Minh Nhat Nguyen, Licheng Liu, et al. Ragen: Understanding self-evolution in llm agents via multi-turn reinforcement learning. arXiv preprint arXiv:2504.20073, 2025.

[65] Ronald J Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. Machine learning, 8(3):229–256, 1992.

[66] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation. arXiv preprint arXiv:2308.08155, 2023.

[67] Tianbao Xie, Danyang Zhang, Jixuan Chen, et al. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In Advances in Neural Information Processing Systems Datasets and Benchmarks Track, 2024.

[68] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. arXiv preprint arXiv:2309.03409, 2023.

[69] John Yang, Carlos E Jimenez, Alex L Zhang, Kilian Lieret, Joyce Yang, Xindi Wu, Ori Press, Niklas Muennighoff, Gabriel Synnaeve, Karthik R Narasimhan, et al. Swe-bench multimodal: Do ai systems generalize to visual software domains? arXiv preprint arXiv:2410.03859, 2024.

[70] Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. Webshop: Towards scalable real-world web interaction with grounded language agents. In Advances in Neural Information Processing Systems, 2022.

[71] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. arXiv preprint arXiv:2406.12045, 2024.

[72] Haoran Ye, Xuning He, Vincent Arak, Haonan Dong, and Guojie Song. Meta context engineering via agentic skill evolution. arXiv preprint arXiv:2601.21557, 2026.

[73] Lei Yin, Wentao Cheng, Zhida Qin, Tianyu Huang, Yidong Li, and Gangyi Ding. Autoue: Automated generation of 3d games in unreal engine via multi-agent systems. arXiv preprint arXiv:2603.07106, 2026.

[74] Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. Advances in Neural Information Processing Systems, 38:113222–113244, 2026.

[75] Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Xian Li, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. Self-rewarding language models. arXiv preprint arXiv:2401.10020, 2024.

[76] Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. Textgrad: Automatic differentiation via text. arXiv preprint arXiv:2406.07496, 2024.

[77] Chaoyun Zhang, Liqun Li, Shilin He, Xu Zhang, Bo Qiao, Si Qin, Minghua Ma, Yu Kang, Qingwei Lin, Saravan Rajmohan, et al. Ufo: A ui-focused agent for windows os interaction. In Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), pages 597–622, 2025.

[78] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, et al. Aflow: Automating agentic workflow generation. In International Conference on Learning Representations, volume 2025, pages 34040–34077, 2025.

[79] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, et al. Agentic context engineering: Evolving contexts for self-improving language models. In International Conference on Learning Representations, volume 2026, pages 86069–86100, 2026.

[80] Wei Zhang, Jian Yang, Renshuai Tao, Linzheng Chai, Shuyue Guo, Jiajun Wu, Xiaoming Chen, Ganqu Cui, Ning Ding, Xander Xu, et al. V-gamegym: Visual game generation for code large language models. In Findings of the Association for Computational Linguistics: ACL 2026, pages 5613–5641, 2026.

[81] Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, et al. Webarena: A realistic web environment for building autonomous agents. In International Conference on Learning Representations, volume 2024, pages 15585–15606, 2024.

[82] Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. Large language models are human-level prompt engineers. In International Conference on Learning Representations, 2023.

Appendices

A Detailed Introduction of Datasets

This appendix details the four game-generation benchmarks used in the experiments of Section 6.1, one per subsection, together with the scoring protocols we apply to each: GameCraft-Bench [32], GameDev-Bench [5], V-GameGym [80], and VeriGame [15]. For every dataset we describe how the benchmark is constructed, how candidates are validated, and the role it plays in our evaluation. Inside the loop we do not rely on any dataset's public evaluator, using our own rubrics for validation and feedback to avoid leakage; the three LLM-judge rubrics of Section A.3 are that self-constructed protocol.

A.1 GameCraft-Bench

GameCraft-Bench [32] asks the agent to produce a runnable Godot game from a task description: 140 tasks spanning 15 game families, such as platformers, strategy, and roguelikes. Each task pairs a public specification with a staged starter project, and the agent must edit scenes, scripts, and resources until the intended game actually plays. Evaluation replays recorded input traces inside a Godot runtime and scores hidden rubric categories spanning mechanics, depth, visuals, and art. Because scoring replays the trace in the real engine, a candidate that imports but never responds to the recorded inputs earns no credit. GameCraft-Bench is the primary instantiation of the executable-gated principle in our experiments, the setting of the harness and rubric analyses in Section 7, and the source of the dynamic-rubric instantiations in Table 7.

A.2 GameDev-Bench

GameDev-Bench [5] raises the difficulty to 132 multi-file Godot development tasks derived from web and video tutorials, covering gameplay logic, 2D and 3D graphics, and user-interface work. Each task combines code with multimodal assets, such as shaders, sprites, and animations, and the average solution requires more than three times the lines of code and file changes of typical software-engineering benchmarks. Every candidate must remain importable and pass deterministic execution-based checks, so a solution receives no credit unless it loads in the Godot editor and satisfies the task's runtime assertions. The benchmark is organized into four sub-types, 2D Graphics & Animation (2DGA), 3D Graphics & Animation (3DGA), User Interface, and Gameplay Logic, which Table 3 reports separately, and its tutorial-derived tasks are where the silent code-level failures of the case studies in Section B are most prominent.

A.3 V-GameGym

V-GameGym [80] contains 2,219 Pygame game samples across 100 thematic clusters, each refactored over a fixed recording horizon so that code, screenshots, and video are all scored at once. The agent receives a short game description and must produce a Pygame project whose recorded session shows playable behavior. Because the official channels, Code, Screenshot, and Video, are graded together over the same recording, a static code patch that never renders visible gameplay receives no credit. Our in-loop protocol adds a Build Health (BH) rubric, defined below, which reports whether the project runs at all.

Inside the loop we score V-GameGym tasks with three LLM-judge rubrics of our own, defined below, and we do not rely on the benchmark's public evaluator for validation or feedback, which avoids leakage. The same Visual Usability and Intent Alignment rubrics also score the VeriGame columns of Table 4, complementing that benchmark's verifier chain and its Build Health channel, while the reported V-GameGym numbers in that table keep the official Code/Screenshot/Video channels.

Build Health. Whether a generated project builds and serves. The artifact must be structurally complete, launch without critical errors, and stay runnable under headless execution; the judge inspects the project structure and runs the entry point, so a project that fails to build or serve receives a markedly depressed score, keeping build failures low across a healthy corpus. Build Health is the legality gate in the A/B admission of Section 4.3 and the only one of the three that is hard.

Visual Usability. Whether the rendered game is visibly playable, judged from screenshots captured during runtime: scenes are coherent, animated, and interactable. When no screenshot is available, the judge infers visual usability from code and UI clues, such as scene composition, layout, and draw calls, instead of defaulting to a neutral score.

Intent Alignment. The degree to which the implemented mechanism matches the intent expressed in the task instruction. The judge decomposes the instruction into requirements and checks executable behavior against each one, so a project that only resembles the intended game without exposing the requested mechanic scores low. This is the criterion behind the interface-versus-intent failures in the case studies (Cases #7 and #8).

A.4 VeriGame

VeriGame [15] evaluates web games on the JavaScript/TypeScript/HTML stack, covering 100 games across seven genres. Instead of replaying recorded input, the benchmark injects runtime state directly into the running game and runs a verifier chain over the exposed gameplay variables, asserting that the intended mechanics act on them. The chain therefore measures behavior through the game's own state, which makes it well suited to catching the interface-versus-intent failures reported in Cases #7 and #8 of Section B. Build Health is the benchmark's hard channel, reporting whether the web build loads and serves. For the VeriGame columns of Table 4, we keep the verifier chain and Build Health and add the Visual Usability and Intent Alignment rubrics of Section A.3, which let the loop grade visible playability and intent matching for mechanics the verifier chain does not assert.

B More Case Studies

This appendix presents the remaining case studies from the four benchmarks, each contrasting the baseline output with the Game-Evolver output.

B.1 Rubric Cases

Dynamic rubrics are not fixed checklists: they are regenerated for each epoch from the current harness and the active task description, so their dimensions track what the harness can actually observe. Table 7 shows three instantiations from the GameCraft-Bench evolution.

The first two rows keep the dataset fixed (idle-dungeon-guild) and vary the harness. Under the process-oriented harness, which centers on edit-verify handoff and probe-first repair, the rubric is process-centric (workflow coherence, repair quality, exploration efficiency, skill selection, tool/MCP effectiveness, context compilation), with playtest signal quality and feature progress visibility as the only player-facing dimensions. Under the evidence-gathering harness, which adds dependency checks and file-tree inspection so it observes the candidate project itself, the rubric shifts to the artifact: public feature completion, core gameplay depth, interaction correctness, progression and end state, playability and balance, runtime feedback quality, and demo coverage.

The last two rows hold the harness fixed and vary the dataset. The seven outcome dimensions survive intact for visualnovel-grimfable, with anchors localized to the task, such as the UI feedback emphasized by the visual novel against the level progression emphasized by the dungeon game. Dynamic rubrics thus inherit the evidence structure of the harness and specialize it to the active task, while the hard rubrics of the admission gate (Section 4.3) stay fixed, so legality never moves with the re-drafted soft criteria.

harness	Dynamic Rubric
`gcbench / idle-dungeon-guild` *harness summary: process-oriented inner harness. It anchors the public task spec, summarizes probe outputs and recent failures, requires edit-verify handoff, enforces workspace boundaries, runs Godot headless playtests, runtime smoke tests, demo replay audits, artifact inventory, entrypoint discovery, checkpoint rollback, and probe-first repair workflow.*	`workflow_coherence`: Multi-step workflow (plan → edit → verify) is coherent and avoids thrashing. `playtest_signal_quality`: Runtime/playtest observations meaningfully informed edits (task family: Godot game crafting with deterministic input replay evidence). `feature_progress_visible`: Player-visible mechanics progressed toward the public task goal. `repair_quality`: Failures were repaired without regressing previously working behavior. `exploration_efficiency`: Agent explored alternatives efficiently under harness budget discipline. `skill_selection_usefulness`: Skill loading/curator modules improved decision quality on this task. `tool_mcp_effectiveness`: Tool/MCP interfaces produced actionable evidence before patching. `context_compilation_clarity`: Context compilation kept relevant history and trimmed noise.
`gcbench / idle-dungeon-guild` *harness summary: stronger evidence-gathering harness. It keeps the same task anchoring, probe summaries, failure history, workspace boundary, Godot headless playtest, runtime smoke, regression suite, checkpoint rollback, and probe-first workflow, but adds tool_dependency_check and mcp_file_tree_inspect to inspect candidate project structure and dependency state before/while repairing gameplay.*	`public_feature_completion`: Observable gameplay implements the important requirements in the public task instruction, including level progression and UI feedback; nominal files, labels, or planned features without executable behavior receive no credit. `core_gameplay_depth`: The core game loop has meaningful state transitions, consequences, and enough mechanical depth to sustain play in this Godot game-crafting task with deterministic input replay evidence. `interaction_correctness`: Real player inputs trigger correct, deterministic gameplay responses, and the runtime evidence shows those responses rather than only a launch or title screen. `progression_and_end_state`: The game supports coherent progression and reaches task-appropriate success, failure, result, or completion states through actual play. `playability_and_balance`: The game is practically playable: controls, pacing, challenge, resources, and failure recovery do not block or trivialize the intended experience. `runtime_feedback_quality`: Runtime presentation clearly communicates actions, state changes, hazards, progress, and outcomes through readable visual or textual feedback. `demo_coverage`: Completed deterministic replay evidence covers the core loop and important states; demo JSON or nominal scenarios without successful replay receive no credit.
`gcbench / visualnovel-grimfable` *harness summary: same evidence-gathering harness as above, but applied to a different public task. It anchors the visual-novel task spec, uses file-tree inspection, dependency checks, Godot replay evidence, demo replay audits, and rollback-based verification to guide repair.*	`public_feature_completion`: Observable gameplay implements the important requirements in the public task instruction, including UI feedback; nominal files, labels, or planned features without executable behavior receive no credit. `core_gameplay_depth`: The core game loop has meaningful state transitions, consequences, and enough mechanical depth to sustain play in this Godot game-crafting task with deterministic input replay evidence. `interaction_correctness`: Real player inputs trigger correct, deterministic gameplay responses, and the runtime evidence shows those responses rather than only a launch or title screen. `progression_and_end_state`: The game supports coherent progression and reaches task-appropriate success, failure, result, or completion states through actual play. `playability_and_balance`: The game is practically playable: controls, pacing, challenge, resources, and failure recovery do not block or trivialize the intended experience. `runtime_feedback_quality`: Runtime presentation clearly communicates actions, state changes, hazards, progress, and outcomes through readable visual or textual feedback. `demo_coverage`: Completed deterministic replay evidence covers the core loop and important states; demo JSON or nominal scenarios without successful replay receive no credit.

Table 7 Dynamic rubrics formed from different datasets and harness focus.

B.2 Game Cases

The following cases illustrate the changes in game quality before and after evolution.

Case #3: Tycoon Pirate Port from GameCraft-Bench. In the baseline trace of Figure 10, building is available but no raid action or settlement state is shown, and the final frame is labeled incomplete and stuck. The evolved trace retains building, adds a raid-target selection screen, and reaches a settlement screen with statistics. The visible comparison therefore supports a narrower claim than the original wording: the evolved artifact closes an interaction path that is absent from the displayed baseline run.

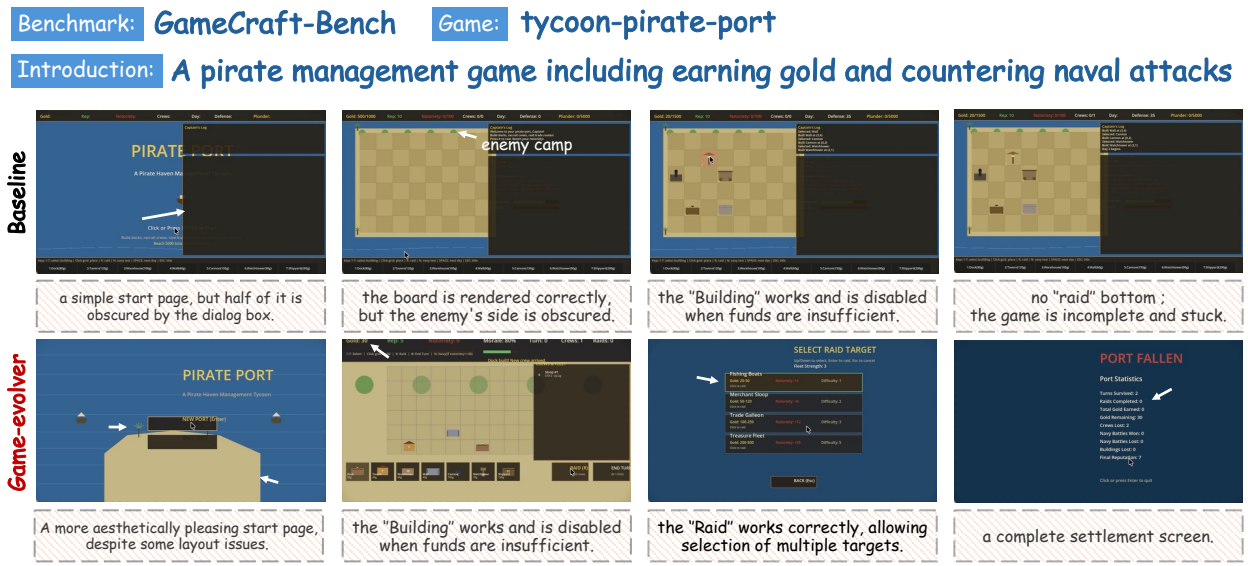

Figure 10 Case #3: tycoon-pirate-port from GameCraft-Bench. This is a pirate management game in which players build a port, earn gold, raid multiple targets, and reach a settlement state. The images above are screenshots of games generated by the baseline, and it scored 0.3250; the images below are screenshots of games generated by Game-Evolver, and it scored 0.4483.

Case #4: Water Depth Scene from GameDev-Bench. The GameDev-Bench failure in Figure 11 is visual rather than structural: the water scene is present, but the baseline camera preview does not frame all five green spheres. The evolved workspace shows the corrected camera framing and receives a score of 1.0. The figure directly supports the camera-framing correction; it does not expose the internal verification step that selected the new pose.

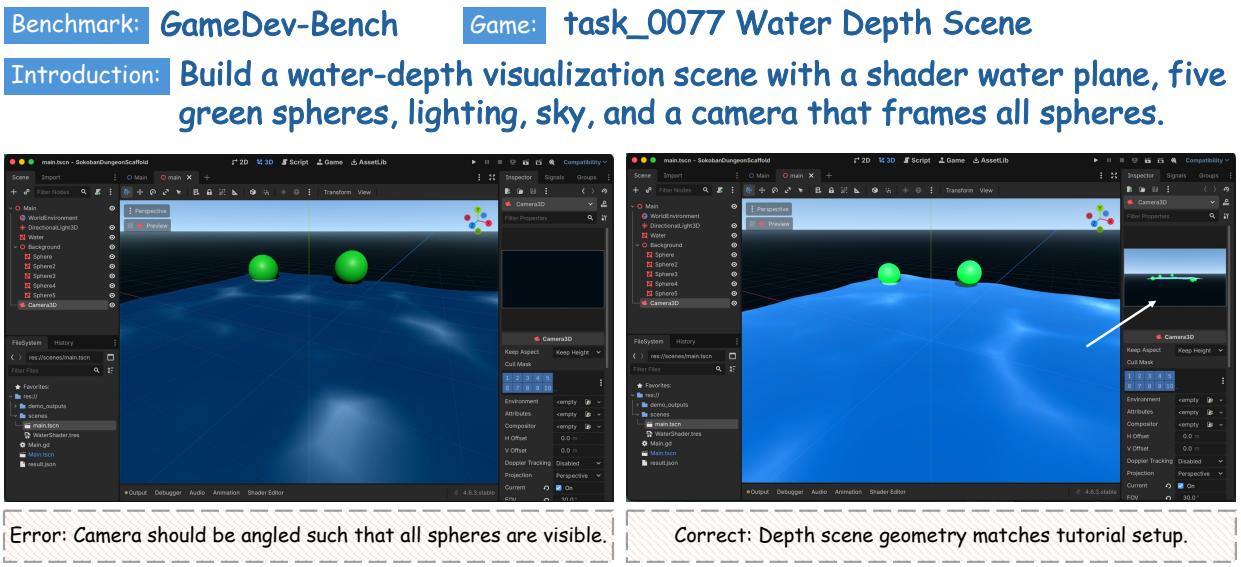

Figure 11 Case #4: task_0077 Water Depth Scene from GameDev-Bench. The agent is asked to build a water-depth visualization scene with a shader water plane, five green spheres, lighting, sky, and a camera that frames all spheres. The left image is the Godot workspace of the baseline (score: 0.0); the right image shows Game-Evolver (score: 1.0).

Case #5: SpaceStoneDodger from V-GameGym. The baseline trace in Figure 12 starts directly in gameplay and shows a controllable spaceship, but its later frames contain a key-state `TypeError`. The evolved trace adds start and options pages, retains controllable gameplay, and reaches a game-over screen. The screenshots thus distinguish early renderability from completion of a playable recorded session.

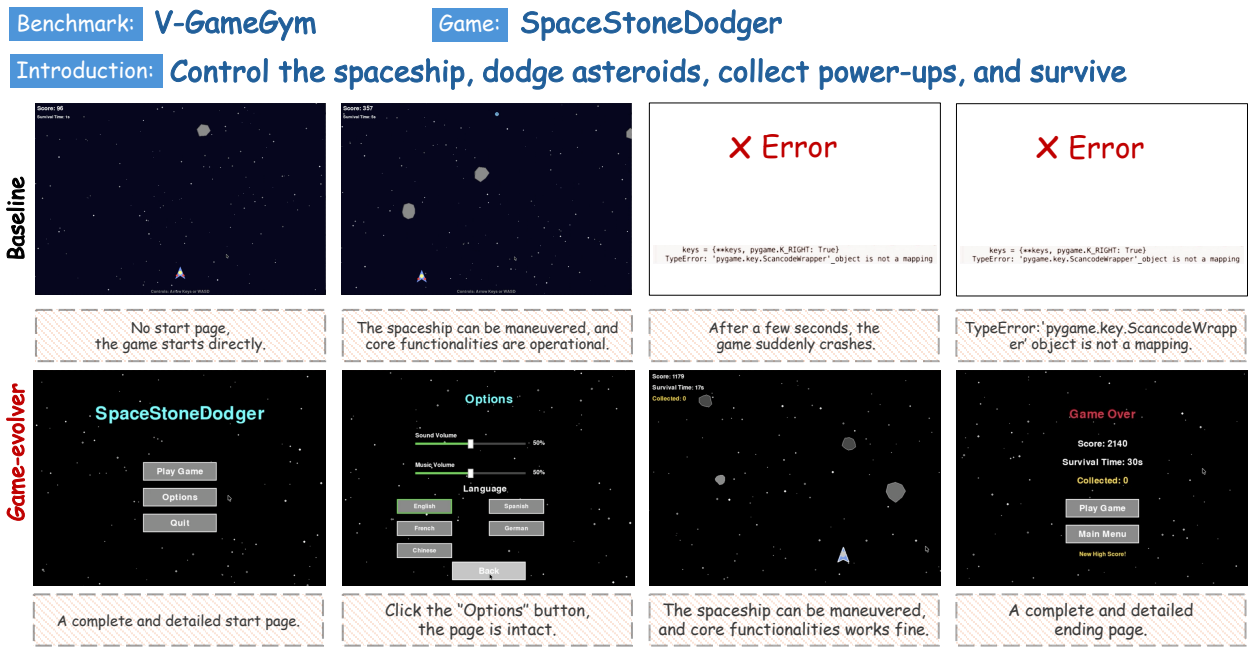

Figure 12 Case #5: SpaceStoneDodger from V-GameGym. The player controls the spaceship to dodge asteroids and collect power-ups. The images above are screenshots of games generated by the baseline, and it scored 0.1980; the images below are screenshots of games generated by Game-Evolver, and it scored 0.3520.

Case #6: Dark Tower Maze from V-GameGym. Figure 13 shows that the baseline renders a maze and moves the explorer automatically, but the annotated trace states that the player cannot control it and that the route demonstration ends after waiting. In the evolved trace, a recorded button press moves the explorer, and later frames reach Levels 2 and 3. The figure therefore supports the change from an automated demonstration to

player-controlled progression.

Figure 13 Case #6: Dark Tower Maze from V-GameGym. This is a top-down maze exploration whose goal is to advance through the levels. The images above are screenshots of games generated by the baseline, and it scored 0.3260; the images below are screenshots of games generated by Game-Evolver, and it scored 0.4470.

Case #7: Crossy Road from VeriGame. The baseline trace in Figure 14 already lets the bird cross the road and reaches an ending page, but it begins without a start page and its later scene layout changes abruptly. The evolved trace adds a start page, preserves controllable crossing, extends the scene as the bird moves upward, and reaches an ending page. The visible difference is therefore improved flow and scene continuity, not the creation of movement or an ending that the baseline entirely lacked.

Figure 14 Case #7: Crossy Road from VeriGame. The player controls the bird to move upward and dodge cars. The left images are screenshots generated by the baseline, and the right images are those generated by Game-Evolver.

Case #8: Dwarf Fortress from VeriGame. The baseline trace in Figure 15 presents the game interface, but its annotations show that Adventure/Look is inert, a dwarf cannot be placed, and the run only waits for an enemy. The evolved trace places a dwarf through the Create Dwarf action, shows a stage event, and changes the Z-axis to manage another level. These are directly visible state changes rather than interface-only differences.

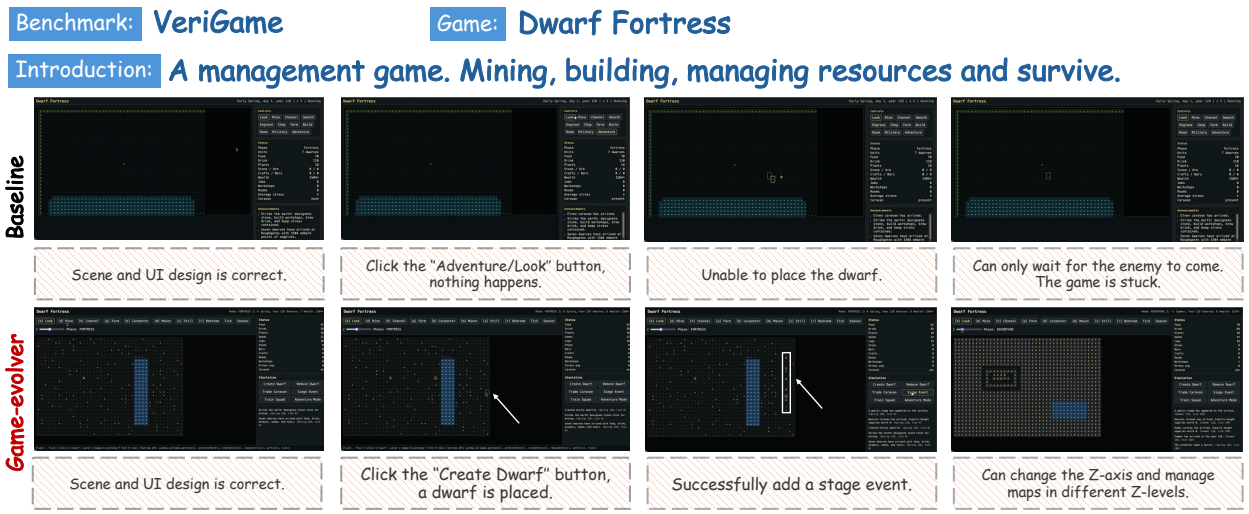

Figure 15 Case #8: Dwarf Fortress from VeriGame. A management game: mine, build, manage resources, and survive. The upper images are screenshots generated by the baseline, and the lower images are those generated by Game-Evolver.

C Application to Real Game Projects

To examine use beyond curated benchmarks, we apply Game-Evolver to godot-tiny-mmo [3], an open-source multiplayer game project built with Godot. Unlike the benchmark tasks, the project already runs, so the target is a feature-level extension rather than recovery of a broken scaffold. Figure 16 directly shows two pieces of evidence: the code diff adds boss-state update logic, and the after frame contains a boss-health bar beneath the dungeon title that is absent from the before frame. We apply the same matched-replay admission schema used elsewhere, but that feedback comes from the framework record rather than from the screenshot itself. The case therefore demonstrates a visible feature addition on the maintained project without claiming that the image alone verifies the full internal evolution trace.

[3] https://github.com/SlayHorizon/godot-tiny-mmo

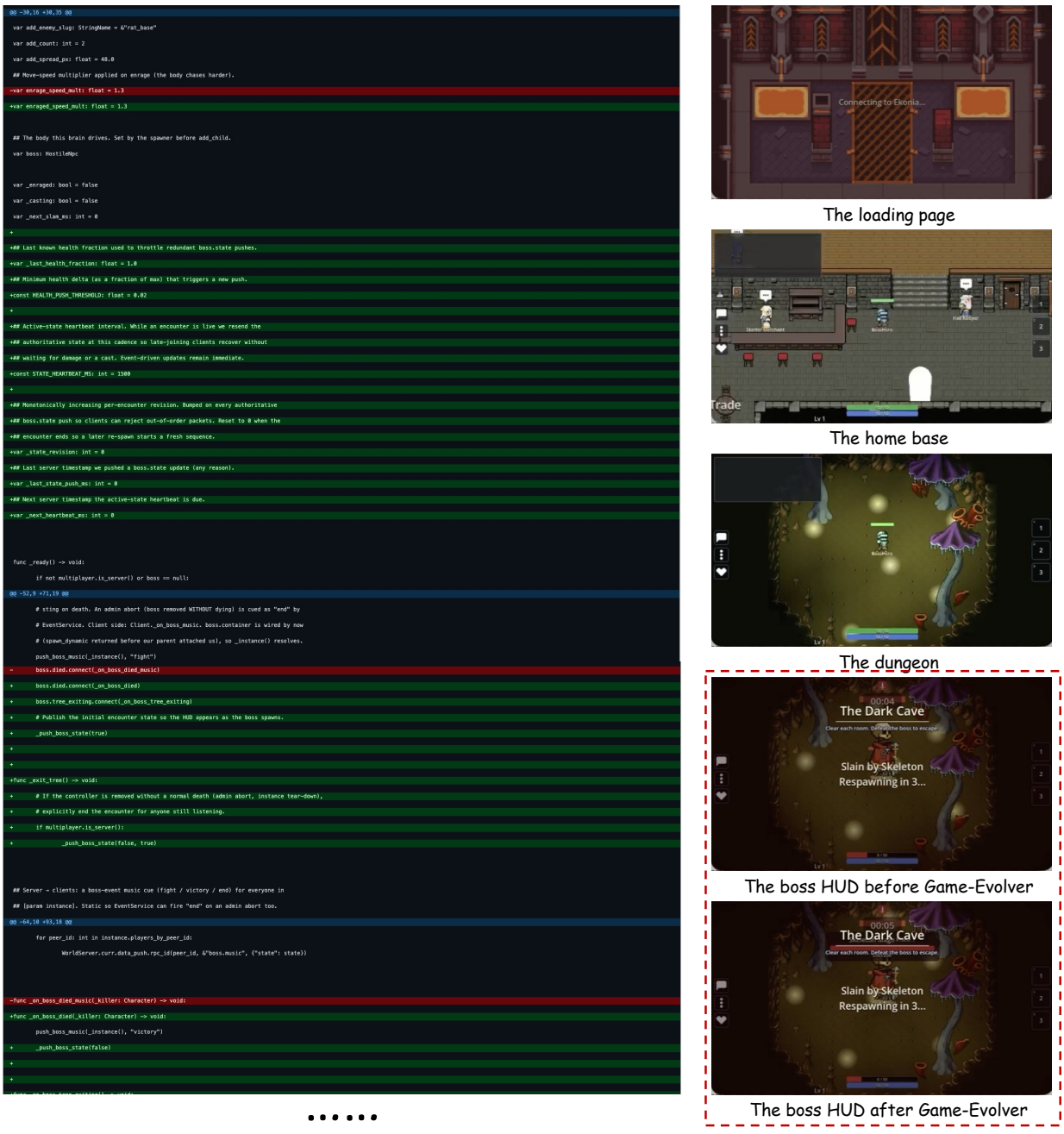

Figure 16 Application of Game-Evolver to an open-source Massively Multiplayer Online (MMO) game. The code diff adds boss-state updates, and the before/after frames show the resulting boss-health bar beneath the dungeon title.